

1. 编译与运行

```
1. int x = 42; // (1)
int incre(int x) { // (2)
    return x + 1;
}
int main(void) {
    int x; // (3) 未定义
    x = 15;
    if (x == 15) {
        int x = incre(10);
        printf("%d\n", x++); // (4) 输出 11, x 变成 12
    }
    printf("%d\n", incre(x)); // (5) 输出 16
    return 0;
}
```

1.4 编译过程详解

阶段	工具	输入	输出	说明
预处理	cpp	.c/.cpp	预处理后的源码	#include, #define 等
编译	ccl/cclp	预处理后的源码	汇编代码.s	语法分析、代码优化
汇编	as	.s	目标文件.o/.obj	生成机器码
链接	ld	.o 文件	可执行文件	链接库函数等

1.5 常用 gcc/g++ 选项

```
g++ main.cpp -o main.exe -Wall -Wextra
选项 说明
-o <file> 指定输出文件名
-c 只编译不链接, 生成.o 文件
-g 生成调试信息
-Wall 开启所有常见警告
-Wextra 开启额外警告
-std=c++17 使用 C++17 标准
-O0/-O1/-O2/-O3 优化级别 (0=无优化, 3=最高优化)
-Idir 添加头文件搜索路径
-Ldir 添加库文件搜索路径
-llib 链接指定库
```

2. 基础语法与变量

2.2 main 函数的合法签名

```
int main(void) { ... } // 方式一
int main(int argc, char *argv[]) { ... } // 方式二 (命令行参数)
// 其他方式实现定义的方式 (不推荐)
// 重要: 可以省略 return 0; (仅对 main 函数), 编译器会自动添加。
```

2.4 C 是静态类型语言

2.5 算术类型总结

2.5.1 整数类型

类型	最小宽度	通常宽度	有符号范围 (通常)	无符号范围 (通常)
short	16 bits	16 bits	-32,768 ~ 32,767	0 ~ 65,535
int	16 bits	32 bits	-2 ³¹ -1 ~ 2 ³¹ -1	0 ~ 2 ³² -1
long	32 bits	32/64 bits	平台相关	平台相关
long long	64 bits	64 bits	-2 ⁶³ -1 ~ 2 ⁶³ -1	0 ~ 2 ⁶⁴ -1

重要关系: sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)

2.5.2 浮点类型

类型	最小精度	说明	格式 (通常)
float	单精度		IEEE754 binary32
double	双精度		IEEE754 binary64
long double	扩展精度	至少和 double 一样好	

选择建议: 默认用 double, 不要用浮点类型做整数运算。

2.5.3 字符类型

类型	范围	说明
signed char	-128 ~ 127	有符号字符
unsigned char	0 ~ 255	无符号字符
char	实现定义	可能是 signed 或 unsigned

```
// ASCII 码关键值
'0' == 48
'9' == 57
'A' == 65
'Z' == 90
'a' == 97
'z' == 122
// 字符转大小写
char to_uppercase(char x) {
    return x - ('a' - 'A'); // 避免 magic number
}
```

2.5.4 布尔类型 (C99 起)

```
#include <stdbool.h> // C99 起需要 (C23 起不需要)
bool flag = true; // true == 1
bool other = false; // false == 0
// C99 前用 int 模拟
int is_lowercase(char c) {
    return c >= 'a' && c <= 'z'; // 返回 1 或 0
}
```

2.5.5 转义序列

转义序列	含义
\\n	换行
\\t	水平制表符
\\r	回车
\\b	退格
\\f	换页符
\\c	单引号
\\d	双引号
\\\\	反斜杠

2.6 格式符

类型	scanf
short	%d
int	%d
long	%ld
long long	%lld
unsigned short	%u
unsigned	%u
unsigned long	%lu
unsigned long long	%llu
float	%f
double	%lf (输出可 %f)
long double	%Lf
char	%c
char* (字符串)	%s

```
// scanf 必须加 & (取地址)
int a;
scanf("%d", &a); // 正确
printf("%d + %d = %d\n", a, b, a + b);
```

3. 运算符与控制流

3.1 算术运算符

% b 是取模 (取整数)

```
// 整数除法: 向零截断 (C99/C++11 起)
int a = 7 / 2; // a == 3
int b = -7 / 2; // b == -3 (向零截断)
// 浮点除法
double c = 7.0 / 2; // c == 3.5
// 取模恒等式: (a / b) * b + (a % b) == a
// 取模符号: 结果符号与被除数 a 相同
// 3.2 类型转换 (整数提升)
// 隐式转换 (整数提升)
double pi = 3.14;
int diameter = 20;
double c = pi * diameter; // int 自动转为 double
// 显式转换
int x = (int)3.14; // C 风格
```

```
double d = double(3.14); // C++ 函数风格
3.4 自增/自减运算符
形式 行为 返回值
++a a 先加 1 返回新值
a++ a 后加 1 返回旧值
--a a 先减 1 返回旧值
a-- a 后减 1 返回旧值
int x = 42;
int a = ++x; // x = 43, a = 43
int b = x++; // b = 43, x = 44
3.5 整数溢出
// 有符号整数溢出 = 未定义行为 (UB) !
int max = 2147483647; // 231 - 1
int overflow = max + 1; // UB! 可能变成负数、崩溃、或 其他
// 无符号整数溢出 = 安全 (模运算)
unsigned int umax = 4294967295U; // 232 - 1
unsigned int uw = umax + 1; // uw == 0 (回绕)
```

3.7 逻辑运算符

运算符	含义
!	非
&&	与 (短路求值)
	或 (短路求值)

3.8 位运算符 (Bitwise Operators)

运算符	名称	说明
-	按位取反	0 变 1, 1 变 0
&	按位与	两位都为 1 才为 1
	按位或	有一位为 1 就为 1
^	按位异或	两位不同才为 1
<<	左移	所有位向左移动
>>	右移	所有位向右移动

```
// 基本位运算
unsigned char a = 0b11001100; // 204
unsigned char b = 0b10101010; // 170
unsigned char c = a & b; // 0b10001000 = 136
unsigned char d = a | b; // 0b11111110 = 238
unsigned char e = a ^ b; // 0b01100110 = 102
unsigned char f = ~a; // 0b00111001 = 51
```

```
// 左移 <<
unsigned char g = a << 1; // 0b10011000 = 152
// 规则: 左边丢弃, 右边补 0
// 11001100 << 1 = 10011000
```

```
// 右移 >>
unsigned char h = a >> 1; // 0b01100110 = 102
// 无符号数: 左边补 0, 右边丢弃
// 有符号数: 左边补符号位 (算术右移) 或补 0 (逻辑右移), 取决于实现
```

循环左移/右移 (Circular/Rotate Shift)

```
// 循环左移 (Rotate Left): 移出的位补到另一边
// 例: 10001 (17) 循环左移 1 位 -> 00011 (3)
// 解释: 110001 -> 000111 -> 00011
unsigned int rotate_left(unsigned int x, unsigned int n) {
    unsigned int bits = sizeof(x) * 8; // 总位数
    n = n % bits; // 处理 n > bits 的情况
    return (x << n) | (x >> (bits - n));
}
// 循环右移 (Rotate Right)
// 例: 10001 (17) 循环右移 1 位 -> 11000 (24)
// 解释: 100011 -> 110001 -> 11000
unsigned int rotate_right(unsigned int x, unsigned int n) {
    unsigned int bits = sizeof(x) * 8;
    n = n % bits;
    return (x >> n) | (x << (bits - n));
}
```

```
// 示例
unsigned int val = 0b10001; // 17
unsigned int r1 = rotate_left(val, 1); // 0b00011 = 3
unsigned int r = rotate_right(val, 1); // 0b11000 = 24
// 常见位运算技巧
// 设置第 n 位为 1
x |= (1u << n);
// 设置第 n 位为 0
x &= ~(1u << n);
// 翻转第 n 位
x ^= (1u << n);
// 检查第 n 位是否为 1
bool is_set = (x >> n) & 1u;
// 或者 bool is_set = x & (1u << n);
```

```
// 取最低位的 1
lowbit = x & (-x); // 或 x & (-x + 1)
// 清除最低位的 1
x &= (x - 1);
// 判断是否为 2 的幂
bool is_power_of_2 = (x & (x - 1)) == 0 && x != 0;
```

3.9 控制流

```
while (n--) { // 执行 n 次
    // break: 终止循环
}
// continue: 跳过本次循环的剩余部分
while (std::cin >> x) { // 读到 EOF 或失败时停止
    for (int i = 0; i < n; ++i) { ... }
    for (int i = n; i >= 0; --i) { ... }
}
// 危险: unsigned 的无限循环
for (unsigned i = n; i >= 0; --i) { ... } // 死循环! unsigned 永远 >= 0
// 范围 for (C++11)
for (auto &x : container) { ... }
switch
switch (x) {
    case 1: // ...
        break; // 不要忘记 break!
    case 2: // ...
        break;
    default: // ...
}
```

```
3.10 逗号运算符
int a = (1, 2, 3); // a = 3, 逗号运算符返回最后一个值
// 优先级最低, 经常需要加括号
4.1 指针与引用
4.1.1 指针基础
int x = 42;
int *p = &x; // & 取地址运算符
printf("%d\n", *p); // * 解引用运算符, 输出 42
*p = 100; // 通过指针修改 x
// 错误示例: 未初始化指针
// int *p;
// *p = 10; // UB! p 是野指针
```

```
4.2 指针运算
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr; // p 指向 arr[0]
*(p + 1); // arr[1] == 20
*(p + 2); // arr[2] == 30
p++; // p 现在指向 arr[1]
```

```
// p[n] 等价于 *(p + n)
p[0]; // arr[0]
p[1]; // arr[2]
p[-1]; // arr[0] (有效!)
4.3 常见指针错误
// 错误1: 未初始化指针
int *p;
*p = 10; // UB: p 是野指针
// 错误2: 空指针解引用
int *p = NULL;
*p = 10; // UB (可能段错误)
// 错误3: 悬空指针
int *p;
{ // 内部作用域
    int x = 42;
    p = &x;
}
// x 在这里销毁
// UB: p 指向已销毁的对象
*p = 10;
// 正确做法
int *p = new int(42); // 动态分配
*p = 10;
delete p; // 记得释放!
```

```
4.4 指针与 const
const int x = 42;
int y = 100;
const int *p1 = &x; // p1 指向 const int (不能通过 p1 修改)
int const *p2 = &x; // 同上 (等价写法)
int *const p3 = &y; // p3 是 const 指针 (不能改变指向)
const int *const p4 = &x; // p4 是 const 指针, 指向 const int
// int *ref = &x; // 编译错误, 反过来可以
4.5 引用 (C++11)
int x = 42;
int &ref = x; // ref 是 x 的引用 (别名)
ref = 100; // x 也变成 100
ref = y; // x = 200, ref 仍然引用 x (不能重新绑定)
```

```
// 引用的本质 (编译器视角)
// int &ref = x; 等价于 *const ref = &x;
// 即 ref 是一个不能改变指向的指针, 自动解引用
4.6 指针 vs 引用
特性 指针 索引
是否需要初始化 可以稍后赋值 必须立即初始化
能否为 null 可以 不可以
能否重新指向 可以 不可以
语法 需要解引用 *p 直接使用 ref
存在性 可能是编译器的实现细节 现细节
```

```
4.7 函数指针
// 函数指针类型
int (*fp)(int, int); // fp 是指向接受两个 int 返回 int 的函数的指针
// fp 指向 max 函数
fp = &max; // & 可以省略
// 调用
int result = (*fp)(a, b); // * 可以省略
int result = fp(a, b); // 等价
```

5. 数组与字符串

```
5.1 数组
int arr[5]; // 声明 5 个 int 的数组 (未初始化)
int arr[5] = {1,2,3,4,5}; // 初始化
int arr[] = {1,2,3}; // 长度自动推断为 3
int arr[] = {1,2}; // arr[0]=1, arr[1]=2, 其余为 0
// 数组作为函数参数会退化为指针
void foo(int arr[5]); // 实际上等价于 void foo(int *arr);
// 数组大小
int n = sizeof(arr) / sizeof(arr[0]); // C 方式求元素个数 (仅对数组有效)
// C++ 推荐方式
template <typename T, std::size_t N>
constexpr std::size_t array_size(T (&)[N]) { return N; }
```

```
5.2 多维数组
int matrix[3][4]; // 3x4 矩阵
int matrix[2][3] = {{1,2,3}, {4,5,6}};
// 函数参数
void foo(int m[][4]); // 必须指定除第一维外的所有维度
void bar(int (*m)[4]); // 等价写法 (数组指针)
// 行优先存储
// matrix[i][j] 的地址 = matrix + i * 4 + j
5.3 C 风格字符串
char str[] = "hello"; // 包含 \\0 结尾, 共 6 字节, 可以修改
char str[6] = "hello"; // 同上, 拷贝构造
char *str = "hello"; // 指向字符串常量
// 可以写作 const char *str = "hello"; 更安全, 避免误修改
// str[0] = 'H'; // UB! 字符串常量不可修改
// str = "world"; // 合法, str 可以指向其他字符串常量
// 字符串函数 (<cstring>)
#include <cstring>
sizeof(str); // 大小 (含 \\0)
strlen(str); // 长度 (不含 \\0)
strcmp(s1, s2); // 比较 (<0, 0, >0) == 比较地址
strcpy(dst, src); // 复制
strncpy(dst, src, n); // 复制最多 n 个字符
strcat(dst, src); // 拼接 (注意缓冲区溢出!)
```

```
// 更安全的方式
strcpy(dst, src, sizeof(dst)); // BSD (非标准 C/C++)
5.4 C++11 std::string
#include <string>
// 构造
std::string s1; // 空字符串
std::string s2 = "hello"; // C 字符串构造
std::string s3(s2); // 拷贝构造
std::string s4 = s2.substr(1, 3); // "ell" (从位置 1 开始, 取 3 个字符)
```

```
// 常用操作
s.empty(); // 是否为空
s.size(); // 长度 (0(1))
s.length(); // 同上
s[n]; // 第 n 个字符 (不检查越界)
s.at(n); // 第 n 个字符 (越界抛异常)
s.c_str(); // 返回 C 风格字符串 (const char*)
s.find(sub); // 查找子串, 返回位置或 string::npos
s.find(sub, pos); // 从 pos 开始查找
```

```
// 比较
s1 == s2; // 内容比较 (不是指针比较!)
s1 < s2; // 字典序比较
```

```
// 拼接
s3 = s1 + s2; // 返回新字符串
s1 += s2; // 追加
// 插入/删除
s.insert(pos, sub);
s.erase(pos, len);
// 转换为其他类型
std::to_string(42); // "42"
std::stoi("42"); // 42 (string to int)
std::stod("3.14"); // 3.14
5.5 std::string_view (C++17)
#include <string_view>
// string_view 不拥有数据, 只是观察已有数据
std::string str = "hello";
std::string_view sv = str; // 从 string 构造
std::string_view sv2 = "world"; // 从字符串常量构造
// 返回新的 string_view (0(1), 不拷贝数据)
sv.substr(1, 3);
// string_view 引用的数据必须在 string_view 存活期间保持有效
```

6. 函数详解

6.2 参数传递方式

方式	语法	是否修改原值	拷贝开销
传值	void f(int x)	修改 x 不影响原值	有拷贝
传引用	void f(int &x)	修改 x 会影响原值	无拷贝
传 const 引用	void f(const int &x)	不能修改原值	无拷贝

```
void by_value(int x) { x = 100; } // 原值不变
void by_pointer(int *x) { *x = 100; } // 原值改变
void by_reference(int &x) { x = 100; } // 原值改变
void by_const_ref(const BigObject &obj);
// C 方式: 通过指针参数
void minmax(const int *arr, int n, int *min, int *max) {
    // C++ 方式1: 返回 pair
    std::pair<int, int> minmax(const std::vector<int> &v) {
        auto [m, mx] = minmax(v); // C++17 结构化绑定
    }
// C++ 方式2: 返回 tuple
std::tuple<int, double, char> foo();
auto [a, b, c] = foo();
// C++ 方式3: struct/类
struct Result { int min; int max; };
Result minmax(const std::vector<int> &v);
// C++ 方式4: 通过引用参数返回
void minmax(const std::vector<int> &v, int &min, int &max) {
    // 原值改变
}
6.4 Lambda 表达式 (C++11)
// 语法: [捕获] (参数) -> 返回类型 { 函数体 }
// 完整形式
auto add = [](int a, int b) -> int { return a + b; };
// 省略返回类型 (编译器自动推导)
auto add = [](int a, int b) { return a + b; };
// 捕获
int x = 10;
auto f1 = [x]() { return x; }; // 值捕获 (拷贝原值)
auto f2 = [&x]() { return ++x; }; // 引用捕获 (修改原值)
auto f3 = [=]() { return x; }; // 值捕获所有外部变量
auto f4 = [&]() { return ++x; }; // 引用捕获所有外部变量
auto f5 = [x, &y]() { ... }; // 混合捕获
auto f6 = [x]() mutable { return ++x; }; // mutable 允许修改捕获的拷贝
```

```
// 泛型 lambda (C++14)
auto add = [](auto a, auto b) { return a + b; };
// 在算法中使用
std::sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; }); // 降序排序
```

7.1 内存管理

```
7.1.1 内存布局
+-----+-----+ 高地址
| 栈区 (Stack) | 局部变量、函数参数 (向下增长)
+-----+-----+
| 堆区 (Heap) | 动态分配 (向上增长)
+-----+-----+
| 全局/静态区 | 全局变量、static 变量 (Data Segment)
+-----+-----+
| 代码区 (Text) | 程序代码
+-----+-----+ 低地址
```

7.2 栈内存 (Stack)

```
void foo() {
    int x; // 栈上分配, 自动管理
    int arr[100]; // 栈上数组
} // 离开作用域自动释放
```

```
// 特点:
// - 自动分配/释放
// - 大小有限 (通常几 MB)
// - 分配速度快
7.3 堆内存 (Heap)
C 方式
#include <stdlib.h>
// malloc: 分配内存, 不初始化
int *p = (int *)malloc(sizeof(int) * n);
// calloc: 分配内存, 清零
int *q = (int *)calloc(n, sizeof(int));
// realloc: 重新分配
p = (int *)realloc(p, new_size);
// 释放
free(p);
p = NULL; // 避免悬空指针
```

```
// 常见错误
int *p = malloc(10); // 错误: 未转换为 int*
// C++ 中不要 malloc
C++11 方式
// new / delete: 单个对象
int *p = new int; // 分配一个 int (未初始化)
int *q = new int(42); // 分配并初始化为 42
int *r = new int(42); // C++11 列表初始化
delete p;
// new[] / delete[]: 数组
int *arr = new int[100]; // 分配 100 个 int (默认初始化)
int *arr2 = new int[100](); // 值初始化 (全部置 0)
int *arr3 = new int[100]{1,2}; // 前两个初始化, 其余置 0
delete[] arr; // 必须用 delete[]!
```

new/delete vs malloc/free

特性	new/delete	malloc/free
类型	运算符	函数
头文件	不需要	<cstdlib>
初始化	会调用构造函数	不初始化
数据	会调用析构函数	不调用析构函数
是否安全	new/delete 是 (自动计算大小)	手动计算大小
类型安全	是 (自动计算大小)	否 (需手动 sizeof)
可以 realloc	否	可以
可以重载	可以	不可以

7.4 智能指针 (C++11)

```
#include <memory>

// unique_ptr: 独占所有权
std::unique_ptr<int> p(new int(42));
std::unique_ptr<int> q = std::make_unique<int>(42);

// C++14 推荐
// q = p; // 错误! 不能拷贝
auto r = std::move(p); // 可以移动

// shared_ptr: 共享所有权 (引用计数)
std::shared_ptr<int> sp1 = std::make_shared<int>(42);
std::shared_ptr<int> sp2 = sp1; // 引用计数 +1
cout << sp1.use_count(); // 输出 2
sp1.reset(); // sp1 释放所有权, 引用计数 -1
cout << sp2.use_count(); // 输出 1

// weak_ptr: 弱引用 (不增加引用计数, 用于打破循环引用)
std::weak_ptr<int> wp = sp1;

// 自动管理数组
std::unique_ptr<int[]> arr(new int[100]);

7.5 常见内存错误
1. 2. 使用已释放的内存 (use-after-free)
delete p;
// UB! p 是悬空指针

// 3. double free
int *p = new int(42);
int *q = p;
delete p;
delete q; // UB! 同一块内存释放两次

// 4. 不匹配的 new/delete
int *p = new int[10];
delete p; // UB! 应该用 delete[]

// 5. 栈空间返回指针
int *bad() {
    int x = 42;
    return &x; // UB! x 在函数返回后销毁
}

// 正确做法: 用智能指针或确保生命周期匹配
std::unique_ptr<int> good() {
    return std::make_unique<int>(42);
}

7.6 RAII (Resource Acquisition Is Initialization) C++ 核心编程范式: 资源获取即初始化
// 原则: 将资源的生命周期绑定到对象的生命周期
// 示例: MutexGuard
class MutexGuard {
    std::mutex &m_mutex;
public:
    explicit MutexGuard(std::mutex &m) : m_mutex(m) {
        m_mutex.lock();
    }
    ~MutexGuard() { m_mutex.unlock(); }
};

void safe_function() {
    MutexGuard guard(my_mutex); // 加锁
    // ... 临界区代码 ...
} // guard 自动解锁, 自动解锁

// C++ 标准库的 RAII 示例:
// - std::ifstream: 构造时打开文件, 析构时关闭
// - std::lock_guard: 构造时加锁, 析构时解锁
// - std::unique_ptr: 构造时获取所有权, 析构时释放内存

8. 类与对象 (C++)
8.1 class vs struct
class struct
默认访问权限 private public
默认继承方式 private public
功能 完全相同 完全相同

class Person {
private:
    std::string name;
    int age;
public:
    void print() const;
};

struct Point {
// 默认 public
    double x, y;
};

8.2 访问控制
访问限定符 类内 子类 外部
private ✓ × ×
protected ✓ ✓ ×
public ✓ ✓ ✓

class Student {
private:
    std::string name; // 私有数据成员
    std::string id;
    int entranceYear;
public:
    void printInfo() const; // 公共接口
    bool isGraduated(int year) const;
};

8.3 this 指针
class Student {
public:
    bool isGraduated(int year) const {
        return year - this->entranceYear >= 4; // 等价于 year - entranceYear
    }
    // 编译器实际处理:
    // bool isGraduated(const Student *this, int year) {
    //     return year - this->entranceYear >= 4;
    // }

// 调用:
Student s;
s.isGraduated(2024);
// 等价于: isGraduated(&s, 2024);

8.4 const 成员函数
class Student {
public:
    bool isGraduated(int year) const; // const 成员函数
    void setId(const std::string &newID); // non-const 成员函数
};

const Student s;
s.isGraduated(2024); // OK: const 对象可以调用 const 成员函数
s.setId("2024"); // 错误! const 对象不能调用 non-const 成员函数

// 规则:
// - const 成员函数的 this 指针类型是 const ClassName*
// - const 成员函数不能修改数据成员
// - const 成员函数不能调用 non-const 成员函数

8.5 Getters and Setters (封装)
class Student {
```

```
std::string name;
std::string id;
int entranceYear;
public:
    // Getter (const 成员函数)
    const std::string &getName() const { return name; }

    int getEntranceYear() const { return entranceYear; }

    // Setter
    void setName(const std::string &newName) { name = newName; }
    void setEntranceYear(int year) {
        if (year < 1900) throw std::invalid_argument("Invalid year");
        entranceYear = year;
    }
};

8.6 类内初始化 (C++11)
struct Point2d {
    double x = 0; // 类内初始化
    double y = 0;
    Point2d() = default; // 使用类内初始化
    Point2d(double x_) : x(x_), y(y_) {} // y 用类内初始化
};

Point2d p; // x=0, y=0 (但注意: p.x 如果是默认初始化可能是未定义值)
Point2d q(3.14); // x=3.14, y=0

成员初始化优先级 (从高到低):
构造函数的初始化列表中的初始化器
类内初始化
默认初始化 (类类型调用默认构造, 内置类型不初始化/值初始化)

9.1 构造函数、析构函数与拷贝控制
class Student {
    std::string name;
    std::string id;
    int entranceYear;
public:
    // 构造函数: 函数名 = 类名, 无返回值
    Student(const std::string &name, const std::string &id, int ey) :
        name(name), id(id), entranceYear(ey) {}
    // 初始化列表

    // 默认构造函数 (无参数)
    Student() = default; // 让编译器合成

    // 或
    Student() : name(""), id(""), entranceYear(0) {}

    // 委托构造 (C++11)
    Student(const std::string &name, const std::string &id,
            : Student(name, id, std::stoi(id_.substr(0, 4))) {}

// 调用方式
Student a("Alice", "2020123123", 2020); // 直接初始化
Student b("Bob", "2020123124"); // 委托构造
Student c; // 默认构造
// 或: 如果 struct 里 int x 则 UB
Student d = a; // 拷贝构造
Student e(a); // 列表初始化

Student f(); // 警告: 这是函数声明, 不是构造对象!
Student g(); // 值初始化 (调用默认构造) // 等价于 Student g = Student();

Best Practice:
• 总是使用初始化列表初始化成员 (比在函数体中赋值高效)
• 初始化顺序与成员声明顺序一致 (不是初始化列表顺序)
• 默认构造函数只有在“默认状态合理”时才提供

9.2 析构函数
class Dynarray {
    std::size_t m_length;
    int *m_storage;
public:
    Dynarray(std::size_t n) : m_length(n), m_storage(new int[n]()) {}

    ~Dynarray() {} // 析构函数: ~类名, 无参数, 无返回值, delete[] m_storage;

    // 或: ~Dynarray() = default; (编译器合成)

// 析构函数自动调用时机:
// - 局部对象离开作用域
// - delete/delete[] 动态对象
// - 成员对象在包含它的对象销毁时

// 析构顺序:
// 1. 执行析构函数体
// 2. 按声明的逆序销毁成员
// 3. 调用父类析构函数

9.3 拷贝控制成员
class Dynarray {
    std::size_t m_length;
    int *m_storage;
public:
    // 1. 拷贝构造函数
    Dynarray(const Dynarray &other) :
        m_length(other.m_length), m_storage(new int[other.m_length]) {
            std::copy_n(other.m_storage, m_length, m_storage);
        }

    // 2. 拷贝赋值运算符
    Dynarray &operator=(const Dynarray &other) {
        if (this != &other) { // 自我赋值检查
            auto new_data = new int[other.m_length];
            std::copy_n(other.m_storage, other.m_length, new_data);
            delete[] m_storage; // 先释放旧资源
            m_storage = new_data;
            m_length = other.m_length;
        }
        return *this;
    }

    // 3. 移动构造函数 (C++11)
    Dynarray(Dynarray &other) noexcept :
        m_length(std::exchange(other.m_length, 0)),
        m_storage(std::exchange(other.m_storage, nullptr)) {}

    // 4. 移动赋值运算符 (C++11)
    Dynarray &operator=(Dynarray &other) noexcept {
        if (this != &other) {
            delete[] m_storage;
            m_length = std::exchange(other.m_length, 0);
            m_storage = std::exchange(other.m_storage, nullptr);
        }
        return *this;
    }

    // 5. 析构函数
    ~Dynarray() { delete[] m_storage; }
};

9.4 Copy-and-Swap 技巧
class Dynarray {
```

```
public:
    void swap(Dynarray &other) noexcept {
        std::swap(m_storage, other.m_storage);
        std::swap(m_length, other.m_length);
    }

    // 拷贝赋值 (通过传值同时获得拷贝和移动赋值)
    Dynarray &operator=(Dynarray other) noexcept {
        swap(other);
        return *this;
    }

    // 传值时: 左值 -> 拷贝构造; 右值 -> 移动构造
    // a = b // 左 a = Dynarray(10); // 右值 a = std::move(b); // 右

9.5 = default 与 = delete
class Example {
public:
    Example() = default; // 显式要求编译器合成
    Example(const Example &) = default; // 使用默认拷贝行为
    Example &operator=(const Example &) = delete; // 禁止拷贝
    Example(Example &&) = delete; // 禁止移动
    ~Example() = default;
};

// 常用用法:
// - 禁止拷贝: 拷贝构造和拷贝赋值都是 delete
// - 只移动类型: 拷贝 = delete, 移动 = default

10. Rule of 3 / Rule of 5 / Rule of 0
10.1 Rule of Three (C++98) 如果一个类需要自定义以下任一函数, 通常三个都需要自定义:
• 析构函数 (destructor)
• 拷贝构造函数 (copy constructor)
• 拷贝赋值运算符 (copy assignment operator)
原因: 通常需要自定义析构函数来管理资源 (如内存), 那就意味着默认的浅拷贝行为是错误的, 必须自定义拷贝构造和拷贝赋值来实现深拷贝。

10.2 Rule of Five (C++11) 扩展为五个函数:
• 析构函数
• 拷贝构造函数
• 拷贝赋值运算符
• 移动构造函数 (move constructor)
• 移动赋值运算符 (move assignment operator)
原则: "Define zero or five of them."

特殊成员函数生成规则
函数 生成条件 不生成 / 删除条件 考试重点
默认构造 T() 用户没写任何构造 用户写了任意构造 -> 不生成 2025 Mid: 写了 T(int) 后 T obj; 编译失败
拷贝构造 T(const T&) 用户没写, 且满足成员/基类条件 成员/基类拷贝构造不可访问 不影响拷贝构造生成
移动构造 T(T&&) 用户没写拷贝/移动/赋值/析构 用户写了任意拷贝/移动/赋值 -> 不生成 一旦手写 copy ctor, move 就没问题了
拷贝赋值 operator= 默认声明 有 const 成员或引用成员 -> delete 2023 Mid #14 核心考点
移动赋值 operator= 默认声明 有 const 成员或引用成员 -> delete 同上
析构 ~T() 用户没写 基类必须 virtual ~Base() = default; delete 时 UB

10.3 Rule of Zero 如果一个类的所有成员都能正确管理自己的资源 (如都用智能指针), 那么不需要自定义任何拷贝控制成员。
// Rule of Zero 示例
class Student {
    std::string name; // string 自己管理内存
    std::vector<int> scores; // vector 自己管理内存
    std::unique_ptr<Address> addr; // unique_ptr 自己管理内存
public:
    // 不需要自定义析构函数、拷贝构造、拷贝赋值等!
    // 编译器合成的默认版本会正确调用成员的对应变操作
};

10.4 深拷贝 vs 浅拷贝
// 浅拷贝 (错误示例)
class Shallow {
    int *data;
public:
    Shallow(const Shallow &other) : data(other.data) {}
    // 问题: 两个对象指向同一块内存!
    // - 析构时会 double delete
    // - 修改会影响另一个对象
};

// 深拷贝 (正确示例)
class Deep {
    class Deep {
        int *data;
        std::size_t len;
    public:
        Deep(const Deep &other) : len(other.len), data(new int[other.len]) {
            std::copy_n(other.data, len, data);
        }
        // 两个对象各自拥有独立的内存拷贝
    };

    // 11. 运算符重载
    11.1 基本规则
    // 语法: operator@, @ 是运算符

    // 成员函数形式 (二元运算符的第一个操作数是 *this)
    class Rational {
        int num, den;
    public:
        Rational operator+(const Rational &rhs) const;
        Rational operator*(const Rational &rhs);
        bool operator==(const Rational &rhs) const;

        // 前缀递增
        Rational &operator++() {
            num += den;
            return *this;
        }

        // 后缀递增 (int 参数用于区分)
        Rational operator++(int) {
            auto tmp = *this;
            ++*this;
            return tmp;
        }
    };

    // 非成员函数形式
    Rational operator+(const Rational &lhs, const Rational &rhs);
    std::ostream &operator<<(std::ostream &os, const Rational &r);
    std::istream &operator>>(std::istream &is, Rational &r);

    11.2 应该作为成员的运算符 • ++ (前缀和后缀)、--
    • * (解引用)、>
    • = (赋值, 必须是成员)
    • +=, -=, *=, /= 等复合赋值
    • () (函数调用)
```

```
• [] (下标)
11.3 建议作为非成员的运算符 • 算术运算符: +, -, *, /
• 比较运算符: ==, !=, <, >, <=, >=
• 流运算符: <<, >>
原因: 如果需要支持左侧操作数的隐式转换, 不能是成员函数。

Rational r(1, 2);
r = 1; // 如果 operator== 是成员: r.operator==(Rational(1)) == OK
1 = r; // 如果 operator== 是成员: 1.operator==(r) 一错误!
// 如果是非成员: operator==(Rational(1), r) == OK

11.4 避免重复的写法
// 比较运算符: 定义 == 和 <, 其余用它们表示
bool operator==(const Rational &lhs, const Rational &rhs) {
    return !(lhs < rhs);
}
bool operator<(const Rational &lhs, const Rational &rhs) {
    return rhs < lhs;
}
bool operator<=(const Rational &lhs, const Rational &rhs) {
    return !(rhs < lhs);
}
bool operator>=(const Rational &lhs, const Rational &rhs) {
    return !(lhs < rhs);
}

// 算术运算符: 定义 += 和一元 -, 其余用它们表示
Rational operator+(const Rational &lhs, const Rational &rhs) {
    return Rational(&lhs) += rhs; // 拷贝 + 复合赋值
}
Rational operator*(const Rational &lhs, const Rational &rhs) {
    return Rational(&lhs) *= rhs;
}
bool operator>=(const Rational &lhs, const Rational &rhs) {
    return !(lhs < rhs);
}

// 算术运算符: 定义 += 和一元 -, 其余用它们表示
Rational operator+(const Rational &lhs, const Rational &rhs) {
    return Rational(&lhs) += rhs; // 拷贝 + 复合赋值
}
Rational operator*(const Rational &lhs, const Rational &rhs) {
    return Rational(&lhs) *= rhs;
}

11.5 函数调用运算符
struct Adder {
    int operator()(int a, int b) const {
        return a + b;
    }
};
Adder add;
int sum = add(2, 3); // add.operator()(2, 3)

11.6 类型转换运算符
struct Rational {
    int n, d;
    explicit operator double() const { // explicit 防止隐式转换
        return static_cast<double>(n) / d;
    }
};
Rational r(3, 2);
double d = static_cast<double>(r); // 必须显式转换
double d2 = r; // 错误: 隐式转换

// Contextual conversions (在 if/while/for 中 explicit 可以隐式转 bool)
if (std::cin) {} // OK: explicit operator bool()

11.7 下标运算符
class Dynarray {
public:
    int &operator[](std::size_t n) { return m_storage[n]; }
    const int &operator[](std::size_t n) const { return m_storage[n]; }
    // const 和 non-const 重载

11.8 箭头运算符
class SharedPtr {
public:
    Object &operator*() const;
    Object *operator->() const {
        return std::addressof(this->operator*());
    }
};
// 保证 p->mem 和 (*p).mem 等价

12. 模板 (Templates)
12.1 函数模板
template <typename T> // 只在下一个函数有用
const T &max(const T &a, const T &b) {
    return a < b ? a : b;
}

// 使用 (编译器自动推断类型)
auto m = max(3, 5); // T = int
auto n = max(3.14, 2.71); // T = double

12.2 类模板
template <typename T>
class Vector {
    T *data;
    std::size_t sz, cap;
public:
    Vector() : data(nullptr), sz(0), cap(0) {}
    void push_back(const T &x);
    T &operator[](std::size_t i) { return data[i]; }
    std::size_t size() const { return sz; }
};

Vector<int> v1; // T = int
Vector<std::string> vs; // T = std::string

12.3 模板实例化
// 隐式实例化: 使用时自动实例化
max(3, 5); // 实例化 max<int>
// 显式实例化
template int max<int>(int, int);
// 依次读取三个值

12.5 重要概念 • 模板不是真正的函数/类, 而是编译器生成代码的指导方案
• 模板参数必须在编译时确定
• 未被使用的成员函数不会被实例化
• 编译错误通常在实例化时才暴露

13. 继承与多态
13.1 基本继承
class Item {
protected:
    std::string name;
    double price;
public:
    Item(const std::string &name, double price) :
        name(name), price(price) {}
    virtual double netPrice(int n) const { return n * price; }
    virtual ~Item() = default; // 基类析构函数必须是 virtual!
};

class DiscountedItem : public Item {
    int minQuantity;
    double discount;
public:
    DiscountedItem(const std::string &name, double price,
                    int minQ, double discount)
```

```
        : Item(name, price), minQuantity(minQ),
        discount(discount) {}
double netPrice(int n) const override {
    if (n >= minQuantity)
        return n * (1 - discount) * price;
    return n * price;
}
};
13.2 访问权限与继承
class Base {
    int private_member; // 类内可访问
protected:
    int protected_member; // 类内 + 子类可访问
public:
    int public_member; // 到处可访问
};
class PublicDerived : public Base {
    // public_member -> public
    // protected_member -> protected
    // private_member 不可访问
};
class PrivateDerived : private Base {
    // public_member -> private
    // protected_member -> private
};
struct vs class 的第二个区别:
• struct 默认继承是 public
• class 默认继承是 private
13.3 虚函数与多态
```

特性	说明
virtual	在基类中声明虚函数
override	在子类中显式标记覆盖 (C++11, 推荐)
动态绑定	通过指针/引用调用时, 根据对象的实际类型调用函数
静态绑定	非虚函数, 根据指针/引用的声明类型调用

```
class Base {
public:
    virtual void foo() { cout << "Base\n"; }
    void bar() { cout << "Base::bar\n"; } // 非虚
};
class Derived : public Base {
public:
    void foo() override { cout << "Derived\n"; } //
    // 覆盖
    void bar() { cout << "Derived::bar\n"; }
    // 隐藏 (不要这样做! )
};
```

```
Base *p = new Derived();
p->foo(); // "Derived" — 动态绑定 (virtual)
p->bar(); // "Base::bar" — 静态绑定 (非虚)
13.4 纯虚函数与抽象类
class Shape {
public:
    virtual double area() const = 0; // 纯虚函数
    virtual ~Shape() = default;
};
```

```
// Shape 是抽象类, 不能实例化
// Shape s; // 错误!
class Circle : public Shape {
    double radius;
public:
    double area() const override { return 3.14159 *
        radius * radius; }
};
```

```
13.5 向上转型与向下转型
// 向上转型 (Upcasting): 总是安全的
Derived d;
Base *pb = &d; // OK
Base &rb = d; // OK
// 向下转型 (Downcasting): 需要运行时检查
derived *pd = dynamic_cast<Derived*>(pb);
if (pd) {
    // 转型成功
} else {
    // pb 实际上不指向 Derived 对象
}
```

```
// 引用版本失败时抛异常
try {
    Derived &rd = dynamic_cast<Derived&>(rb);
} catch (std::bad_cast &e) {
    // 转型失败
}
```

```
13.6 继承中的构造函数与析构
class Derived : public Base {
public:
    // 构造函数: 先调 Base 构造, 再初始化 Derived 成员
    Derived() : Base(), member() {}
    // 拷贝构造: 不要忘记拷贝 Base 部分
    Derived(const Derived &other)
        : Base(other), member(other.member) {}
    // 拷贝赋值
    Derived &operator=(const Derived &other) {
        Base::operator=(other); // 先赋值 Base 部分
        member = other.member; // 再赋值
        Derived &return *this;
    }
    // 析构函数: 先析构 Derived 成员, 再调 Base 析构
    ~Derived() {
        // Derived 的析构...
        // 然后自动调用 ~Base()
    }
};
```

13.7 重要设计原则 • **public 继承** 建模 "is-a" 关系

- 不要以其他方式定义和父类成员同名的成员 (除了 override)
- 不要重新定义继承而来的 non-virtual 函数
- 不要重新定义继承而来的缺省参数值
- 基类析构函数应该是 virtual, 否则通过基类指针 delete 派生类对象会导致 UB

容器	底层结构	随机访问	尾部插入	头部插入	中间插入	适用场景
vector	动态数组	O(1)	O(1)	O(n)	O(n)	默认选择
deque	分段数组	O(1)	O(1)	O(1)	O(n)	双端操作
list	双向链表	O(n)	O(1)	O(1)	O(1)	频繁中间插入/删除
forward_list	单向链表	O(n)	O(1)	不支持	O(1)	单向遍历
array	固定数组	O(1)	不支持	不支持	不支持	大小固定

14. STL 容器

14.1 顺序容器

容器	有序	允许重复	底层	操作复杂度
set	是	否	红黑树	O(log n)
map	是	否 (key)	红黑树	O(log n)
multiset	是	是	红黑树	O(log n)
multimap	是	是	红黑树	O(log n)
unordered_set	否	否	哈希表	O(1) 平均
unordered_map	否	否 (key)	哈希表	O(1) 平均

14.2 关联容器

容器	有序	允许重复	底层	操作复杂度
set	是	否	红黑树	O(log n)
map	是	否 (key)	红黑树	O(log n)
multiset	是	是	红黑树	O(log n)
multimap	是	是	红黑树	O(log n)
unordered_set	否	否	哈希表	O(1) 平均
unordered_map	否	否 (key)	哈希表	O(1) 平均

14.3 vector 详解

```
#include <vector>
// 构造
std::vector<int> v1; // 空
std::vector<int> v2(10); // 10 个元素, 值初始化为 0
std::vector<int> v3(10, 42); // 10 个 42
std::vector<int> v4 = {1, 2, 3}; // 初始化列表
std::vector<int> v5(v4.begin(), v4.end()); // 从迭代器范围
```

```
// 容量管理
v.size(); // 当前元素个数
v.empty(); // 是否为空
v.capacity(); // 不重新分配的前提下最多可容纳的元素数
v.reserve(100); // 预留至少 100 个元素的容量
v.resize(20); // 改变元素个数 (新增元素值初始化为 0)
v.shrink_to_fit(); // 请求释放多余容量
// 元素访问
v[i]; // 第 i 个元素 (不检查越界)
v.at(i); // 第 i 个元素 (越界抛出 out_of_range)
v.front(); // 第一个元素
v.back(); // 最后一个元素
v.data(); // 返回底层数组指针
// 添加/删除
v.push_back(x); // 尾部添加 (拷贝)
v.emplace_back(args...); // 尾部原地构造 (C++11, 更高效)
v.pop_back(); // 删除尾部
v.insert(pos, x); // 在迭代器 pos 前插入
v.erase(pos); // 删除 pos 位置的元素
v.erase(first, last); // 删除 [first, last)
v.clear(); // 清空所有元素
// reserve vs resize
v.reserve(100); // capacity >= 100, size 不变
v.resize(100); // size == 100, capacity 至少 100
14.4 list / forward_list
#include <list>
std::list<int> l;
l.push_back(10); // 尾部
l.push_front(20); // 头部
l.insert(it, 30); // 在 it 前插入
l.erase(it); // 删除 it 指向的元素
#include <forward_list>
std::forward_list<int> fl;
fl.push_front(10); // 只有 push_front
fl.insert_after(it, 20); // 在 it 之后插入
fl.erase_after(it); // 删除 it 之后的元素
// forward_list 没有 push_back, pop_back, size()
14.5 map / unordered_map
#include <map>
std::map<std::string, int> scores;
scores["Alice"] = 90; // 插入或赋值
scores.insert({"Bob", 85}); // 插入 (C++11)
scores.emplace("Charlie", 95); // 原地构造插入
auto it = scores.find("Alice"); // 查找, 返回迭代器或 end()
if (it != scores.end()) {
    std::cout << it->second; // 访问值
}
// 遍历 (按 key 排序)
for (const auto &[name, score] : scores) {
    std::cout << name << " : " << score << "\n";
}
#include <unordered_map>
std::unordered_map<std::string, int> um;
// 用法同 map, 但元素无序, O(1) 平均操作
14.6 统一接口所有容器都支持:
• 拷贝构造、拷贝赋值、移动构造、移动赋值
• c.size(), c.empty()
• c.begin(), c.end()
• c.clear()
• c == c2, c != c2 (比较运算符, 要求元素支持)
15. STL 迭代器与算法
15.1 迭代器基础
std::vector<int> v = {1, 2, 3, 4, 5};
// 迭代器类型
std::vector<int>::iterator it; // 可修改元素的迭代器
std::vector<int>::const_iterator cit; // 只读迭代器
// 获取迭代器
auto begin = v.begin(); // 指向第一个元素
auto end = v.end(); // 指向最后一个元素的下一个位置 (后)
auto cbegin = v.cbegin(); // const_iterator
// 迭代器操作
*it; // 解引用, 获取元素
it+=member; // 访问成员 (等价于 (*it).member)
++it; // 前进 (前缀更高效)
it++; // 前进 (后缀)
it == it2; // 比较
it != it2; // 比较 (通常用 != 而不是 <, 因为更通用)
// 范围: [begin, end) —— 左闭右开
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";
}
// 范围 for (C++11) —— 等价于上面的代码
for (auto &x : v) {
    std::cout << x << " ";
}
15.2 迭代器类别
类别 支持操作 适用容器
InputIterator ++, * (只读一次), ==, != istream_iterator
ForwardIterator InputIterator forward_list, unordered_*
BidirectionalIterator InputIterator list, set, map
RandomAccessIterator BidirectionalIterator vector, deque, array, string
15.3 迭代器辅助函数
#include <iterator>
std::advance(it, n); // it 前进 n 步 (可能为负)
std::distance(it1, it2); // 计算 it1 到 it2 的距离
std::next(it); // 返回 it 的下一个位置
std::prev(it); // 返回 it 的上一个位置
15.4 常用算法
```

```
#include <algorithm>
#include <numeric>
// 不修改序列
std::find(begin, end, value); // 查找, 返回迭代器或 end
std::find_if(begin, end, pred); // 查找满足条件的第一个
std::count(begin, end, value); // 计数
std::count_if(begin, end, pred); // 条件计数
std::for_each(begin, end, func); // 对每个元素执行 func
std::equal(begin1, end1, begin2); // 比较两个范围
// 修改序列
std::copy(src_begin, src_end, dst_begin); // 拷贝 n 个
std::copy_n(src_begin, n, dst_begin); // 拷贝 n 个
std::fill(begin, end, value); // 填充
std::transform(begin, end, dst_begin, func); // 转换
std::replace(begin, end, old_val, new_val); // 替换
std::reverse(begin, end); // 反转
std::unique(begin, end); // 去重 (相邻重复), 返回新 end
// 排序和二分查找 (要求 RandomAccessIterator)
std::sort(begin, end); // 快速排序, O(n log n)
std::sort(begin, end, comp); // 自定义比较器
std::stable_sort(begin, end); // 稳定排序
std::binary_search(begin, end, value); // 二分查找 (返回 bool)
std::lower_bound(begin, end, value); // 第一个 >= value 的位置
std::upper_bound(begin, end, value); // 第一个 > value 的位置
// 集合运算 (要求已排序)
std::merge(b1, e1, b2, e2, dst); // 归并
std::set_union(...); // 并集
std::set_intersection(...); // 交集
std::set_difference(...); // 差集
// 数值运算 (<numeric>)
std::accumulate(begin, end, init); // 求和 (init 为初始值)
std::inner_product(b1, e1, b2, init); // 内积
15.5 算法使用示例
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
// 排序
std::sort(v.begin(), v.end()); // 升序
std::sort(v.begin(), v.end(), std::greater<int>()); // 降序
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
// 查找
auto it = std::find(v.begin(), v.end(), 5);
if (it != v.end()) std::cout << "Found!\n";
// 二分查找 (必须先排序)
bool exists = std::binary_search(v.begin(), v.end(), 5);
// 去重 (先排序, 再去重, 然后 erase)
std::sort(v.begin(), v.end());
auto new_end = std::unique(v.begin(), v.end());
v.erase(new_end, v.end());
// 统计
int cnt = std::count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
15.6 重要提醒
// 算法不修改容器大小!
std::vector<int> a = {1,2,3};
std::vector<int> b;
std::copy(a.begin(), a.end(), b.begin()); // UB! b 为空, 没有空间
// 正确做法: 确保目标容器有足够空间
std::vector<int> b(a.size());
std::copy(a.begin(), a.end(), b.begin());
// 或者使用插入迭代器
std::vector<int> b;
std::copy(a.begin(), a.end(), std::back_inserter(b));
16. STL 适配器与特殊适配器
16.1 容器适配器
适配器 底层默认容器 接口 特点
stack deque push, pop, top LIFO
queue deque push, pop, front, back FIFO
priority_queue vector push, pop, top 堆, 最大元素在顶部
```

```
#include <stack>
std::stack<int> s;
s.push(10);
int x = s.top(); s.pop();
#include <queue>
std::queue<int> q;
q.push(10); int front = q.front(); q.pop();
std::priority_queue<int> pq; // 大顶堆
pq.push(10); int max = pq.top(); pq.pop();
16.2 迭代器适配器
#include <iterator>
// 插入迭代器
std::back_inserter<std::vector<int>>> back_it(v);
// 更简洁的创建方式
*back_it = 42; // 等价于 v.push_back(42)
std::front_inserter(l); // 等价于 l.push_front(x)
std::inserter(c, it); // 等价于 c.insert(it, x)
// 流迭代器
std::istream_iterator<int> in_it(std::cin, in_end);
std::ostream_iterator<int> out_it(std::cout, " ");
// 使用
std::vector<int> v(in_it, in_end); // 从 cin 读取直到 EOF
std::copy(v.begin(), v.end(), out_it); // 输出到 cout, 用空格分隔
// 移动迭代器
std::vector<std::string> src = get_words();
std::vector<std::string> dst;
dst.reserve(src.size());
std::copy(std::make_move_iterator(src.begin()), std::make_move_iterator(src.end()), dst::back_inserter(dst));
// 反向迭代器
for (auto rit = v.rbegin(); rit != v.rend(); ++rit) {
    std::cout << *rit << " "; // 逆序输出
}
16.3 std::pair 与 std::tuple
#include <utility>
// pair
std::pair<int, std::string> p1 = {1, "one"};
auto p2 = std::make_pair(2, "two"); // C++17 前常用
std::pair p3{3, "three"}; // C++17 CTAD
```

```
std::cout << p1.first << ", " << p1.second << "\n";
auto [a, b] = p1; // C++17 结构化绑定
// tuple
#include <tuple>
std::tuple<int, double, char> t{1, 2.5, 'a'};
auto [x, y, z] = t; // 获取第 0 个元素
std::get<0>(t);
17. IOSTream 库
17.1 基本输入输出
#include <ostream>
// 标准流
std::cin; // 标准输入 (istream)
std::cout; // 标准输出 (ostream)
std::cerr; // 标准错误 (ostream, 无缓冲)
std::clog; // 标准日志 (ostream, 有缓冲)
// 输入
int x;
std::cin >> x; // 读取 int
std::string s;
std::cin >> s; // 读取字符串 (到空白字符停止)
cin.ignore(); // 前面会把\n留在输入缓冲区
std::getline(std::cin, s); // 读取一行 (包括空格)
// 输出
std::cout << "Hello" << x << std::endl; // endl = 换行 + 刷新缓冲区
std::cout << "Hello" << '\n'; // 更高效 (不强刷刷新)
// 格式化输出
#include <iomanip>
std::cout << std::setw(10) << x; // 宽度 10
std::cout << std::setprecision(2) << d; // 小数点后 2 位
std::cout << std::fixed << d; // 固定小数表示
std::cout << std::hex << x; // 十六进制输出
17.2 文件流
#include <fstream>
// 读文件
std::ifstream infile("data.txt");
if (!infile) { // 检查是否成功打开
    std::cerr << "Cannot open file!\n";
    return 1;
}
// 写文件
std::ofstream outfile("output.txt");
outfile << "Hello, World!\n";
// 打开模式
std::ofstream file("log.txt", std::ios::app); // 追加模式
// 其他模式: in, out, binary, trunc, ate
// RAII: 文件流在销毁时自动关闭
17.3 字符串流
#include <sstream>
// 字符串输出流
std::ostringstream oss;
oss << "The answer is " << 42;
std::string result = oss.str(); // "The answer is 42"
// 字符串输入流
std::istringstream iss("3.14 42 hello");
double d; int n; std::string s;
iss >> d >> n >> s; // d=3.14, n=42, s="hello"
// 类型转换
int x = std::stoi("42"); // string 到 int
double y = std::stod("3.14"); // string 到 double
std::string str = std::to_string(42); // int 到 string
17.4 流状态
std::cin >> x;
if (std::cin) { // 成功
    // ...
} else if (!std::cin) { // 失败
    std::cin.clear(); // 清除错误标志
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // 忽略剩余输入
}
// 检查 EOF
while (std::cin >> x) { // 在 EOF 或输入错误时停止
    // ...
}
18. DynamicArray / Dynarray 完整示例
这是课程的核心示例, 完整展示了 Rule of Five 的所有要素:
#include <algorithm>
#include <cstring>
#include <stdexcept>
#include <utility>
class Dynarray {
private:
    std::size_t m_length; // 数组长度
    int *m_storage; // 指向动态分配的内存
public:
    // 1. 构造函数: n 个 value-initialized 的 int
    explicit Dynarray(std::size_t n)
        : m_length(n), m_storage(new int[n]{} ) {}
    // 2. 默认构造函数: 空数组
    Dynarray() : Dynarray(0) {}
    // 3. 拷贝构造函数 (深拷贝)
    Dynarray(const Dynarray &other)
        : m_length(other.m_length), m_storage(new int[other.m_length] ) {
            std::copy_n(other.m_storage, m_length, m_storage);
        }
    // 4. 移动构造函数
    Dynarray(Dynarray &&other) noexcept
        : m_length(std::exchange(other.m_length, 0)),
          m_storage(std::exchange(other.m_storage, nullptr)) {}
    // 5. 拷贝赋值运算符 (copy-and-swap 技巧)
    Dynarray &operator=(Dynarray other) noexcept {
        swap(other);
        return *this;
    }
    // 6. 析构函数
    ~Dynarray() { delete[] m_storage; }
    // 7. swap 函数
    void swap(Dynarray &other) noexcept {
        using std::swap;
        swap(m_length, other.m_length);
        swap(m_storage, other.m_storage);
    }
    // 8. 元素访问
    int &at(std::size_t n) {
        if (n >= m_length)
            throw std::out_of_range("Dynarray::at");
        return m_storage[n];
    }
};
```

```
Base *p = new Derived();
p->foo(); // "Derived" — 动态绑定 (virtual)
p->bar(); // "Base::bar" — 静态绑定 (非虚)
13.4 纯虚函数与抽象类
class Shape {
public:
    virtual double area() const = 0; // 纯虚函数
    virtual ~Shape() = default;
};
// Shape 是抽象类, 不能实例化
// Shape s; // 错误!
class Circle : public Shape {
    double radius;
public:
    double area() const override { return 3.14159 *
        radius * radius; }
};
13.5 向上转型与向下转型
// 向上转型 (Upcasting): 总是安全的
Derived d;
Base *pb = &d; // OK
Base &rb = d; // OK
// 向下转型 (Downcasting): 需要运行时检查
derived *pd = dynamic_cast<Derived*>(pb);
if (pd) {
    // 转型成功
} else {
    // pb 实际上不指向 Derived 对象
}
// 引用版本失败时抛异常
try {
    Derived &rd = dynamic_cast<Derived&>(rb);
} catch (std::bad_cast &e) {
    // 转型失败
}
13.6 继承中的构造函数与析构
class Derived : public Base {
public:
    // 构造函数: 先调 Base 构造, 再初始化 Derived 成员
    Derived() : Base(), member() {}
    // 拷贝构造: 不要忘记拷贝 Base 部分
    Derived(const Derived &other)
        : Base(other), member(other.member) {}
    // 拷贝赋值
    Derived &operator=(const Derived &other) {
        Base::operator=(other); // 先赋值 Base 部分
        member = other.member; // 再赋值
        Derived &return *this;
    }
    // 析构函数: 先析构 Derived 成员, 再调 Base 析构
    ~Derived() {
        // Derived 的析构...
        // 然后自动调用 ~Base()
    }
};
13.7 重要设计原则 • public 继承 建模 "is-a" 关系
• 不要以其他方式定义和父类成员同名的成员 (除了 override)
• 不要重新定义继承而来的 non-virtual 函数
• 不要重新定义继承而来的缺省参数值
• 基类析构函数应该是 virtual, 否则通过基类指针 delete 派生类对象会导致 UB
14. STL 容器
14.1 顺序容器
容器 底层结构 随机访问 尾部插入 头部插入 中间插入 适用场景
vector 动态数组 O(1) O(1) O(n) O(n) 默认选择
deque 分段数组 O(1) O(1) O(1) O(n) 双端操作
list 双向链表 O(n) O(1) O(1) O(1) 频繁中间插入/删除
forward_list 单向链表 O(n) O(1) 不支持 O(1) 单向遍历
array 固定数组 O(1) 不支持 不支持 不支持 大小固定
14.2 关联容器
容器 有序 允许重复 底层 操作复杂度
set 是 否 红黑树 O(log n)
map 是 否 (key) 红黑树 O(log n)
multiset 是 是 红黑树 O(log n)
multimap 是 是 红黑树 O(log n)
unordered_set 否 否 哈希表 O(1) 平均
unordered_map 否 否 (key) 哈希表 O(1) 平均
14.3 vector 详解
#include <vector>
// 构造
std::vector<int> v1; // 空
std::vector<int> v2(10); // 10 个元素, 值初始化为 0
std::vector<int> v3(10, 42); // 10 个 42
std::vector<int> v4 = {1, 2, 3}; // 初始化列表
std::vector<int> v5(v4.begin(), v4.end()); // 从迭代器范围
// 容量管理
v.size(); // 当前元素个数
v.empty(); // 是否为空
v.capacity(); // 不重新分配的前提下最多可容纳的元素数
v.reserve(100); // 预留至少 100 个元素的容量
v.resize(20); // 改变元素个数 (新增元素值初始化为 0)
v.shrink_to_fit(); // 请求释放多余容量
// 元素访问
v[i]; // 第 i 个元素 (不检查越界)
v.at(i); // 第 i 个元素 (越界抛出 out_of_range)
v.front(); // 第一个元素
v.back(); // 最后一个元素
v.data(); // 返回底层数组指针
// 添加/删除
v.push_back(x); // 尾部添加 (拷贝)
v.emplace_back(args...); // 尾部原地构造 (C++11, 更高效)
v.pop_back(); // 删除尾部
v.insert(pos, x); // 在迭代器 pos 前插入
v.erase(pos); // 删除 pos 位置的元素
v.erase(first, last); // 删除 [first, last)
v.clear(); // 清空所有元素
// reserve vs resize
v.reserve(100); // capacity >= 100, size 不变
v.resize(100); // size == 100, capacity 至少 100
14.4 list / forward_list
#include <list>
std::list<int> l;
l.push_back(10); // 尾部
l.push_front(20); // 头部
l.insert(it, 30); // 在 it 前插入
l.erase(it); // 删除 it 指向的元素
#include <forward_list>
std::forward_list<int> fl;
fl.push_front(10); // 只有 push_front
fl.insert_after(it, 20); // 在 it 之后插入
fl.erase_after(it); // 删除 it 之后的元素
// forward_list 没有 push_back, pop_back, size()
14.5 map / unordered_map
#include <map>
std::map<std::string, int> scores;
scores["Alice"] = 90; // 插入或赋值
scores.insert({"Bob", 85}); // 插入 (C++11)
scores.emplace("Charlie", 95); // 原地构造插入
auto it = scores.find("Alice"); // 查找, 返回迭代器或 end()
if (it != scores.end()) {
    std::cout << it->second; // 访问值
}
// 遍历 (按 key 排序)
for (const auto &[name, score] : scores) {
    std::cout << name << " : " << score << "\n";
}
#include <unordered_map>
std::unordered_map<std::string, int> um;
// 用法同 map, 但元素无序, O(1) 平均操作
14.6 统一接口所有容器都支持:
• 拷贝构造、拷贝赋值、移动构造、移动赋值
• c.size(), c.empty()
• c.begin(), c.end()
• c.clear()
• c == c2, c != c2 (比较运算符, 要求元素支持)
15. STL 迭代器与算法
15.1 迭代器基础
std::vector<int> v = {1, 2, 3, 4, 5};
// 迭代器类型
std::vector<int>::iterator it; // 可修改元素的迭代器
std::vector<int>::const_iterator cit; // 只读迭代器
// 获取迭代器
auto begin = v.begin(); // 指向第一个元素
auto end = v.end(); // 指向最后一个元素的下一个位置 (后)
auto cbegin = v.cbegin(); // const_iterator
// 迭代器操作
*it; // 解引用, 获取元素
it+=member; // 访问成员 (等价于 (*it).member)
++it; // 前进 (前缀更高效)
it++; // 前进 (后缀)
it == it2; // 比较
it != it2; // 比较 (通常用 != 而不是 <, 因为更通用)
// 范围: [begin, end) —— 左闭右开
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";
}
// 范围 for (C++11) —— 等价于上面的代码
for (auto &x : v) {
    std::cout << x << " ";
}
15.2 迭代器类别
类别 支持操作 适用容器
InputIterator ++, * (只读一次), ==, != istream_iterator
ForwardIterator InputIterator forward_list, unordered_*
BidirectionalIterator InputIterator list, set, map
RandomAccessIterator BidirectionalIterator vector, deque, array, string
15.3 迭代器辅助函数
#include <iterator>
std::advance(it, n); // it 前进 n 步 (可能为负)
std::distance(it1, it2); // 计算 it1 到 it2 的距离
std::next(it); // 返回 it 的下一个位置
std::prev(it); // 返回 it 的上一个位置
15.4 常用算法
```

```
#include <algorithm>
#include <numeric>
// 不修改序列
std::find(begin, end, value); // 查找, 返回迭代器或 end
std::find_if(begin, end, pred); // 查找满足条件的第一个
std::count(begin, end, value); // 计数
std::count_if(begin, end, pred); // 条件计数
std::for_each(begin, end, func); // 对每个元素执行 func
std::equal(begin1, end1, begin2); // 比较两个范围
// 修改序列
std::copy(src_begin, src_end, dst_begin); // 拷贝 n 个
std::copy_n(src_begin, n, dst_begin); // 拷贝 n 个
std::fill(begin, end, value); // 填充
std::transform(begin, end, dst_begin, func); // 转换
std::replace(begin, end, old_val, new_val); // 替换
std::reverse(begin, end); // 反转
std::unique(begin, end); // 去重 (相邻重复), 返回新 end
// 排序和二分查找 (要求 RandomAccessIterator)
std::sort(begin, end); // 快速排序, O(n log n)
std::sort(begin, end, comp); // 自定义比较器
std::stable_sort(begin, end); // 稳定排序
std::binary_search(begin, end, value); // 二分查找 (返回 bool)
std::lower_bound(begin, end, value); // 第一个 >= value 的位置
std::upper_bound(begin, end, value); // 第一个 > value 的位置
// 集合运算 (要求已排序)
std::merge(b1, e1, b2, e2, dst); // 归并
std::set_union(...); // 并集
std::set_intersection(...); // 交集
std::set_difference(...); // 差集
// 数值运算 (<numeric>)
std::accumulate(begin, end, init); // 求和 (init 为初始值)
std::inner_product(b1, e1, b2, init); // 内积
15.5 算法使用示例
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2, 6};
// 排序
std::sort(v.begin(), v.end()); // 升序
std::sort(v.begin(), v.end(), std::greater<int>()); // 降序
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });
// 查找
auto it = std::find(v.begin(), v.end(), 5);
if (it != v.end()) std::cout << "Found!\n";
// 二分查找 (必须先排序)
bool exists = std::binary_search(v.begin(), v.end(), 5);
// 去重 (先排序, 再去重, 然后 erase)
std::sort(v.begin(), v.end());
auto new_end = std::unique(v.begin(), v.end());
v.erase(new_end, v.end());
// 统计
int cnt = std::count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
15.6 重要提醒
// 算法不修改容器大小!
std::vector<int> a = {1,2,3};
std::vector<int> b;
std::copy(a.begin(), a.end(), b.begin()); // UB! b 为空, 没有空间
// 正确做法: 确保目标容器有足够空间
std::vector<int> b(a.size());
std::copy(a.begin(), a.end(), b.begin());
// 或者使用插入迭代器
std::vector<int> b;
std::copy(a.begin(), a.end(), std::back_inserter(b));
16. STL 适配器与特殊适配器
16.1 容器适配器
适配器 底层默认容器 接口 特点
stack deque push, pop, top LIFO
queue deque push, pop, front, back FIFO
priority_queue vector push, pop, top 堆, 最大元素在顶部
```

```
#include <stack>
std::stack<int> s;
s.push(10);
int x = s.top(); s.pop();
#include <queue>
std::queue<int> q;
q.push(10); int front = q.front(); q.pop();
std::priority_queue<int> pq; // 大顶堆
pq.push(10); int max = pq.top(); pq.pop();
16.2 迭代器适配器
#include <iterator>
// 插入迭代器
std::back_inserter<std::vector<int>>> back_it(v);
// 更简洁的创建方式
*back_it = 42; // 等价于 v.push_back(42)
std::front_inserter(l); // 等价于 l.push_front(x)
std::inserter(c, it); // 等价于 c.insert(it, x)
// 流迭代器
std::istream_iterator<int> in_it(std::cin, in_end);
std::ostream_iterator<int> out_it(std::cout, " ");
// 使用
std::vector<int> v(in_it, in_end); // 从 cin 读取直到 EOF
std::copy(v.begin(), v.end(), out_it); // 输出到 cout, 用空格分隔
// 移动迭代器
std::vector<std::string> src = get_words();
std::vector<std::string> dst;
dst.reserve(src.size());
std::copy(std::make_move_iterator(src.begin()), std::make_move_iterator(src.end()), dst::back_inserter(dst));
// 反向迭代器
for (auto rit = v.rbegin(); rit != v.rend(); ++rit) {
    std::cout << *rit << " "; // 逆序输出
}
16.3 std::pair 与 std::tuple
#include <utility>
// pair
std::pair<int, std::string> p1 = {1, "one"};
auto p2 = std::make_pair(2, "two"); // C++17 前常用
std::pair p3{3, "three"}; // C++17 CTAD
```

```
std::cout << p1.first << ", " << p1.second << "\n";
auto [a, b] = p1; // C++17 结构化绑定
// tuple
#include <tuple>
std::tuple<int, double, char> t{1, 2.5, 'a'};
auto [x, y, z] = t; // 获取第 0 个元素
std::get<0>(t);
17. IOSTream 库
17.1 基本输入输出
#include <ostream>
// 标准流
std::cin; // 标准输入 (istream)
std::cout; // 标准输出 (ostream)
std::cerr; // 标准错误 (ostream, 无缓冲)
std::clog; // 标准日志 (ostream, 有缓冲)
// 输入
int x;
std::cin >> x; // 读取 int
std::string s;
std::cin >> s; // 读取字符串 (到空白字符停止)
cin.ignore(); // 前面会把\n留在输入缓冲区
std::getline(std::cin, s); // 读取一行 (包括空格)
// 输出
std::cout << "Hello" << x << std::endl; // endl = 换行 + 刷新缓冲区
std::cout << "Hello" << '\n'; // 更高效 (不强刷刷新)
// 格式化输出
#include <iomanip>
std::cout << std::setw(10) << x; // 宽度 10
std::cout << std::setprecision(2) << d; // 小数点后 2 位
std::cout << std::fixed << d; // 固定小数表示
std::cout << std::hex << x; // 十六进制输出
17.2 文件流
#include <fstream>
// 读文件
std::ifstream infile("data.txt");
if (!infile) { // 检查是否成功打开
    std::cerr << "Cannot open file!\n";
    return 1;
}
// 写文件
std::ofstream outfile("output.txt");
outfile << "Hello, World!\n";
// 打开模式
std::ofstream file("log.txt", std::ios::app); // 追加模式
// 其他模式: in, out, binary, trunc, ate
// RAII: 文件流在销毁时自动关闭
17.3 字符串流
#include <sstream>
// 字符串输出流
std::ostringstream oss;
oss << "The answer is " << 42;
std::string result = oss.str(); // "The answer is 42"
// 字符串输入流
std::istringstream iss("3.14 42 hello");
double d; int n; std::string s;
iss >> d >> n >> s; // d=3.14, n=42, s="hello"
// 类型转换
int x = std::stoi("42"); // string 到 int
double y = std::stod("3.14"); // string 到 double
std::string str = std::to_string(42); // int 到 string
17.4 流状态
std::cin >> x;
if (std::cin) { // 成功
    // ...
} else if (!std::cin) { // 失败
    std::cin.clear(); // 清除错误标志
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // 忽略剩余输入
}
// 检查 EOF
while (std::cin >> x) { // 在 EOF 或输入错误时停止
    // ...
}
18. DynamicArray / Dynarray 完整示例
这是课程的核心示例, 完整展示了 Rule of Five 的所有要素:
#include <algorithm>
#include <cstring>
#include <stdexcept>
#include <utility>
class Dynarray {
private:
    std::size_t m_length; // 数组长度
    int *m_storage; // 指向动态分配的内存
public:
    // 1. 构造函数: n 个 value-initialized 的 int
    explicit Dynarray(std::size_t n)
        : m_length(n), m_storage(new int[n]{} ) {}
    // 2. 默认构造函数: 空数组
    Dynarray() : Dynarray(0) {}
    // 3. 拷贝构造函数 (深拷贝)
    Dynarray(const Dynarray &other)
        : m_length(other.m_length), m_storage(new int[other.m_length] ) {
            std::copy_n(other.m_storage, m_length, m_storage);
        }
    // 4. 移动构造函数
    Dynarray(Dynarray &&other) noexcept
        : m_length(std::exchange(other.m_length, 0)),
          m_storage(std::exchange(other.m_storage, nullptr)) {}
    // 5. 拷贝赋值运算符 (copy-and-swap 技巧)
    Dynarray &operator=(Dynarray other) noexcept {
        swap(other);
        return *this;
    }
    // 6. 析构函数
    ~Dynarray() { delete[] m_storage; }
    // 7. swap 函数
    void swap(Dynarray &other) noexcept {
        using std::swap;
        swap(m_length, other.m_length);
        swap(m_storage, other.m_storage);
    }
    // 8. 元素访问
    int &at(std::size_t n) {
        if (n >= m_length)
            throw std::out_of_range("Dynarray::at");
        return m_storage[n];
    }
};
```

```
Base *p = new Derived();
p->foo(); // "Derived" — 动态绑定 (virtual)
p->bar(); // "Base::bar" — 静态绑定 (非虚)
13.4 纯虚函数与抽象类
class Shape {
public:
    virtual double area() const = 0; // 纯虚函数
    virtual ~Shape() = default;
};
// Shape 是抽象类, 不能实例化
// Shape s; // 错误!
class Circle : public Shape {
    double radius;
public:
    double area() const override { return 3.14159 *
        radius * radius; }
};
13.5 向上转型与向下转型
// 向上转型 (Upcasting): 总是安全的
Derived d;
Base *pb = &d; // OK
Base &rb = d; // OK
// 向下转型 (Downcasting): 需要运行时检查
derived *pd = dynamic_cast<Derived*>(pb);
if (pd) {
    // 转型成功
} else {
    // pb 实际上不指向 Derived 
```

```
const int &at(std::size_t n) const {
    if (n >= m_length)
        throw std::out_of_range("Dynarray::at");
    return m_storage[n];
}

int &operator[](std::size_t n) { return m_storage[n]; }
const int &operator[](std::size_t n) const { return m_storage[n]; }

// 9. 容量查询
std::size_t size() const noexcept { return m_length; }
bool empty() const noexcept { return m_length == 0; }

// 10. 迭代器支持 (使 range-based for 可用)
int *begin() noexcept { return m_storage; }
const int *begin() const noexcept { return m_storage; }
int *end() noexcept { return m_storage + m_length; }
const int *end() const noexcept { return m_storage + m_length; }
};
```

// 非成员 swap
void swap(Dynarray &a, Dynarray &b) noexcept {
 a.swap(b);
}

关键设计要点**RAII**: 构造函数分配资源, 析构函数释放资源
深拷贝: 拷贝构造函数和拷贝赋值运算符都创建独立的内存副本
移动语义: 移动操作“窃取”资源, 将被移出对象置于可安全析构的状态
自我赋值安全: copy-and-swap 自动处理
异常安全: copy-and-swap 提供强异常安全保证
noexcept: 移动操作标记 noexcept, 让容器可以高效使用

19. 位运算 (Bitwise Operations) 详解用户特别关注: 位运算在作业中出现过 (循环左移右移等)

19.1 基本运算符回顾

```
// ~ 按位取反 (NOT)
// & 按位与 (AND): 两位都为 1 结果为 1
// | 按位或 (OR): 有一位为 1 结果为 1
// ^ 按位异或 (XOR): 两位不同结果为 1
// << 左移
// >> 右移

unsigned char a = 0b10110011; // 179
unsigned char b = 0b01100110; // 204
```

```
-a; // 0b01001100 = 76
a & b; // 0b10000000 = 128
a | b; // 0b11111111 = 255
a ^ b; // 0b01111111 = 127
```

19.2 移位运算详解左移 *u*:

- 所有位向左移动, 右边补 0
- 等同于乘以 2^n (在无溢出的情况下)
- 移出左边界的位丢弃

```
unsigned char a = 0b00010001; // 17
a << 1; // 0b00100010 = 34 (17 * 2)
a << 2; // 0b01000100 = 68 (17 * 4)
a << 3; // 0b10001000 = 136 (17 * 8)
a << 4; // 0b00010000 = 16 (溢出! 高位 1 被丢弃)
右移 *v*:
```

- 所有位向右移动
 - 无符号数: 左边补 0 (逻辑右移)
 - 有符号数: 左边补符号位 (算术右移, 大多数编译器)
 - 等同于除以 2^n 向下取整
- ```
unsigned char a = 0b10001000; // 136
a >> 1; // 0b01000100 = 68 (136 / 2)
a >> 3; // 0b00010001 = 17 (136 / 8)
```

```
// 有符号数
signed char b = 0b10001000; // -120 (假设 8 位补码)
b >> 1; // 0b11000100 = -60 (补符号位 1)
```

### 19.3 循环移位 (Rotate)

```
// 循环左移: 移出的高位补到低位
// 例: 10001 (17) 循环左移 1 位 -> 00011 (3)
// 步骤: 1|0001 -> 0001 移出 1 -> 00011

unsigned int rotl(unsigned int x, unsigned int n) {
 const unsigned int bits = sizeof(x) * 8;
 n %= bits; // 处理 n >= bits 的情况
 if (n == 0) return x;
 return (x << n) | (x >> (bits - n));
}
```

```
// 循环右移: 移出的低位补到高位
// 例: 10001 (17) 循环右移 1 位 -> 11000 (24)
// 步骤: 1000 11 -> 11000 1 -> 11000
```

```
unsigned int rotr(unsigned int x, unsigned int n) {
 const unsigned int bits = sizeof(x) * 8;
 n %= bits;
 if (n == 0) return x;
 return (x >> n) | (x << (bits - n));
}
```

### 19.4 位运算常用技巧

```
// 1. 检查奇偶
bool is_odd = n & 1; // 最低位为 1 则是奇数
bool is_even = !(n & 1);

// 2. 交换两个数 (不借助临时变量)
a ^= b;
b ^= a;
a ^= b;
```

```
// 3. 判断是否为 2 的幂
bool is_pow2 = (n > 0) && ((n & (n - 1)) == 0);
// 原理: 2 的幂只有一位是 1, n-1 会将该位变为 0, 低位全变 1
```

```
// 4. 计算 2 进制中 1 的个数 (Brian Kernighan 算法)
int count_ones(unsigned int n) {
 int count = 0;
 while (n) {
 n &= n - 1; // 清除最低位的 1
 ++count;
 }
 return count;
}
```

```
// 5. 取最低位的 1
unsigned int lowbit = x & (-x);
// 原理: -x == ~x + 1, 恰好保留了最低位的 1
```

```
// 6. 将最低位的 1 置 0
x &= x - 1;
```

```
// 7. 获取第 n 位
bool get_bit(int x, int n) { return (x >> n) & 1; }
```

```
// 8. 设置第 n 位为 1
void set_bit(int &x, int n) { x |= (1 << n); }
```

```
// 9. 设置第 n 位为 0
void clear_bit(int &x, int n) { x &= ~(1 << n); }
```

```
// 10. 翻转第 n 位
void flip_bit(int &x, int n) { x ^= (1 << n); }
```

20. unsigned 类型详解用户特别关注: unsigned 类型的行为

### 20.1 unsigned 基本性质

```
// unsigned 类型只表示非负数
unsigned int x = 42;
unsigned int y = -1; // 警告: 负数转为 unsigned 会变成很大的数
// y = 4294967295 (假设 32 位)
```

unsigned 的取值范围 (N 位):  $0 \sim 2^N - 1$

```
// unsigned int (32位): 0 ~ 4294967295
// unsigned short (16位): 0 ~ 65535
// unsigned long long (64位): 0 ~ 18446744073709551615
```

### 20.2 unsigned 运算规则

```
// 1. unsigned 运算永远不会溢出, 而是做模运算
unsigned int u = 0;
--u; // u = 4294967295 (环绕)

unsigned int v = 4294967295;
++v; // v = 0 (环绕)

// 2. unsigned + signed 运算时, signed 会被转为 unsigned
unsigned u = 10;
int s = -1;
if (s < u) // 你可能期望 true, 但实际上...
 ; // s 被转为 unsigned int, 变成 4294967295
// 所以 s < u 为 false!
```

```
// 危险示例
for (unsigned i = n; i >= 0; --i) { ... }
// 死循环! i 永远不会 < 0
```

```
// 3. 比较时的陷阱
size_t len = s.size(); // size_t 是 unsigned
if (len < -1) { ... } // 警告: -1 转为 unsigned 会很大
```

### 20.3 unsigned 的使用建议

| 场景      | 建议                                 |
|---------|------------------------------------|
| 数组索引/大小 | 使用 std::size_t (通常是 unsigned long) |
| 位运算     | 使用 unsigned 类型                     |
| 数学计算    | 优先使用 signed (int/long long)        |
| 需要负数的场景 | 绝不能用 unsigned                      |
| 混合运算    | 避免 signed 和 unsigned 混合            |

```
// 安全的做法
std::vector<int> v;
for (std::size_t i = 0; i < v.size(); ++i) { ... }
```

```
// 更安全: 如果不需要索引, 用范围 for
for (const auto &x : v) { ... }

// 如果需要增减遍历 signed 容器
for (int i = static_cast<int>(v.size()) - 1; i >= 0; --i) { ... }
```

21. 初始化方式总结 (值初始化 vs 默认初始化) 用户特别关注: 值初始化和默认初始化的区别

### 21.1 各种初始化方式

```
// ===== C++ 初始化方式 =====

// 1. 默认初始化 (Default Initialization)
int x; // 内置类型: 值不确定 (未定义) !
std::string s; // 类类型: 调用默认构造函数

// 2. 值初始化 (Value Initialization)
int x{}; // 内置类型: 初始化为 0
double d{}; // 0.0
bool b{}; // false
int *p{}; // nullptr
std::string s{}; // 类类型: 调用默认构造函数 (空字符串)
```

```
// 3. 直接初始化
int x(42);
std::string s("hello");
```

```
// 4. 拷贝初始化
int x = 42;
std::string s = "hello";
```

```
// 5. 列表初始化 (Uniform Initialization, C++11)
int x{42}; // 推荐!
int y = {42}; // 等价
std::vector<int> v{1, 2, 3};
```

```
// 6. 聚合初始化
struct Point { int x, y; };
Point p{1, 2}; // 聚合初始化

21.2 关键区别: 默认初始化 vs 值初始化
```

| 类型                | 默认初始化 T x; | 值初始化 T x{};     |
|-------------------|------------|-----------------|
| int, double 等内置类型 | 不初始化! 值不确定 | 初始化为 0          |
| 类类型 (有默认构造)       | 调用默认构造函数   | 调用默认构造函数        |
| 类类型 (无默认构造)       | 编译错误       | 编译错误 (如果无法默认构造) |
| 数组                | 各元素默认初始化   | 各元素值初始化         |
| new T             | 默认初始化      | -               |
| new T{}           | -          | 值初始化            |
| new T{n}          | 各元素默认初始化   | -               |
| new T{n}{}        | -          | 各元素值初始化         |

```
// 示例对比
void foo() {
 int a; // 默认初始化: a 的值不确定! 可能是垃圾值
 int b{}; // 值初始化: b == 0
 int *p1; // 默认初始化: p1 指向不确定位置!
 int *p2{}; // 值初始化: p2 == nullptr

 int *arr1 = new int[10]; // 10 个未初始化的 int
 int *arr2 = new int[10]{}; // 10 个值为 0 的 int
}
```

```
21.3 C++11 初始化列表与构造函数
class MyClass {
 int x;
 std::string s;
public:
 // 初始值列表中的初始化
 MyClass(int x_, const std::string &s_) : x(x_), s(s_) {}

 // 类内初始化
 int default_x = 0; // C++11 类内初始化
};

// 成员初始化优先级:
// 1. 构造函数初始值列表 (最高优先级)
// 2. 类内初始化
// 3. 默认初始化
```

### 21.4 初始化的最佳实践

```
// 1. 总是初始化变量 (用大括号初始化)
int x{}; // 好
int y = 0; // 也好
int z; // 不好! 值不确定

// 2. 类成员用初始值列表
class Good {
 int x;
public:
 Good(int val) : x(val) {} // 好: 直接初始化
};

class Bad {
 int x;
public:
 Bad(int val) { x = val; } // 不好: 先默认初始化, 再赋值
};
```

```
// 3. 动态分配时明确初始化意图
auto p1 = new int; // 不初始化 (注意风险)
auto p2 = new int{}; // 值初始化为 0
auto p3 = new int(42); // 初始化为 42
auto p4 = new int(42); // 同上 (推荐大括号)
```

```
auto arr1 = new int[10]; // 不初始化
auto arr2 = new int[10]{}; // 全部值初始化为 0

22. 常见错误与正确替代写法
22.1 指针与内存相关
// ===== 错误1: 未初始化指针 =====
// 错误代码:
int *p;
*p = 10; // UB: p 是野指针

// 正确写法:
int x = 10;
int *p = &x; // p 指向有效的变量
// 或
int *p = new int(10); // 动态分配
// 使用后: delete p;

// ===== 错误2: 内存泄漏 =====
// 错误代码:
void leak() {
 int *p = new int(42);
 // 没有 delete, 内存泄漏
}

// 正确写法:
void no_leak() {
 auto p = std::make_unique<int>(42); // unique_ptr 自动释放
 // 或使用原始指针时确保 delete
 int *p = new int(42);
 delete p;
}
```

```
// ===== 错误3: 使用已释放的内存 (Use-after-free) =====
// 错误代码:
int *p = new int(42);
delete p;
*p = 100; // UB! p 是悬空指针

// 正确写法:
int *p = new int(42);
{ int *q = p; }
delete p;
p = nullptr; // 释放后置空

// ===== 错误4: Double Free =====
// 错误代码:
int *p = new int(42);
int *q = p;
delete p;
delete q; // UB! 同一块内存释放两次
```

```
// 正确写法:
int *p = new int(42);
{ int *q = p; }
delete p;
p = nullptr; // 释放后置空

// ===== 错误4: Double Free =====
// 错误代码:
int *p = new int(42);
int *q = p;
delete p;
delete q; // UB! 同一块内存释放两次

// 正确写法:
std::unique_ptr<int> p(new int(42));
auto q = p; // 错误: unique_ptr 不能拷贝
auto r = std::move(p); // OK: 转移所有权

// ===== 错误5: 栈变量返回指针 =====
// 错误代码:
int *bad() {
 int x = 42;
 return &x; // UB! x 在函数返回后销毁
}
```

```
// 正确写法:
int *good() {
 return new int(42); // 动态分配
}

// 调用者负责 delete
// ===== 错误6: new[] 与 delete 不匹配 =====
// 错误代码:
int *p = new int[10];
delete p; // 错误! 应该用 delete[]

// 正确写法:
int *p = new int[10];
delete[] p;

// ===== 错误7: 浅拷贝导致 double delete =====
// 错误代码:
class Bad {
 int *data;
public:
 Bad(const Bad &other) : data(other.data) {} // 浅拷贝!
 ~Bad() { delete data; }
```

```
};
Bad a;
Bad b = a; // a 和 b 的 data 指向同一块内存
析构时 double delete!

// 正确写法:
class Good {
 int *data;
public:
 Good(const Good &other) : data(new int(*other.data)) {} // 深拷贝
 ~Good() { delete data; }
```

```
};

22.2 数组与字符串相关
// ===== 错误8: 数组越界 =====
// 错误代码:
int arr[5];
arr[5] = 10; // UB! 越界访问

// 正确写法:
std::vector<int> v(5);
v.at(4) = 10; // at() 会检查越界

// ===== 错误9: 字符串缓冲区溢出 =====
// 错误代码:
char buf[10];
strcpy(buf, "this is too long"); // 缓冲区溢出!

// 正确写法:
std::string s = "this is safe and flexible";
// 或
strncpy(buf, src, sizeof(buf) - 1);
buf[sizeof(buf) - 1] = '\0';
```

```
// ===== 错误10: 修改字符串常量 =====
// 错误代码:
char *s = "hello"; // 字符串常量
s[0] = 'H'; // UB! 试图修改只读内存

// 正确写法:
char s[] = "hello"; // 数组拷贝
s[0] = 'H'; // OK

// 或
std::string s = "hello";
s[0] = 'H'; // OK
```

```
22.3 类与对象相关
// ===== 错误11: 忘记 virtual 析构函数 =====
// 错误代码:
class Base {
public:
 ~Base() {} // 不是 virtual!
};

class Derived : public Base {
 int *data;
public:
 ~Derived() { delete data; } // 不会被调用!
};

Base *p = new Derived();
delete p; // 只调用了 Base::~~Base(), 内存泄漏!
```

```
// 正确写法:
class Base {
public:
 virtual ~Base() = default; // 基类析构函数必须是 virtual
};

// ===== 错误12: 自我赋值问题 =====
// 错误代码:
```

```
class Bad {
public:
 Bad &operator=(const Bad &other) {
 delete data;
 data = new int(*other.data); // 如果 this == &other, UB!
 return *this;
 }
};

// 正确写法 (copy-and-swap):
Bad &operator=(Bad other) {} // 传值 swap(other);
return *this;
}
```

```
// ===== 错误13: 在构造函数中调用虚函数 =====
// 错误代码:
class Base {
public:
 Base() { foo(); } // 危险!
 virtual void foo() {}
};

class Derived : public Base {
 int x;
public:
 Derived() : x(42) {}
 void foo() override { std::cout << x; } // x 还未初始化!
};

// 正确做法: 不要在构造函数/析构函数中调用虚函数
```

```
// ===== 错误14: 构造函数初始值列表顺序错误 =====
// 错误代码:
class Bad {
 int a;
 int b;
public:
 Bad(int x) : b(x), a(b + 1) {} // b 先初始化?
 // 实际上 a 先初始化 (声明顺序), 此时 b 还是未定义!
};

// 正确写法:
class Good {
 int a;
 int b;
public:
 Good(int x) : a(x + 1), b(x) {} // 顺序与声明一致
};

22.4 STL 相关
// ===== 错误15: 迭代器失效 =====
// 错误代码:
std::vector<int> v = {1, 2, 3, 4, 5};
auto it = v.begin() + 2; // 指向 3
v.push_back(6); // OK: 转移所有权
*it = 30; // UB! it 可能已失效

// 正确写法:
auto it = v.begin() + 2;
v.push_back(6);
it = v.begin() + 2; // 重新获取迭代器
*it = 30;
```

```
// ===== 错误16: 对空容器调用 front/back =====
// 错误代码:
std::vector<int> v;
int x = v.front(); // UB!

// 正确写法:
if (!v.empty()) {
 int x = v.front();
}
```

```
// ===== 错误17: 算法修改不存在的元素 =====
// 错误代码:
std::vector<int> a = {1,2,3};
std::vector<int> b;
std::copy(a.begin(), a.end(), b.begin()); // UB! b 为空

// 正确写法:
std::vector<int> b(a.size());
std::copy(a.begin(), a.end(), std::back_inserter(b));
```

```
22.5 类型转换相关
// ===== 错误18: 有符号和无符号混用 =====
// 错误代码:
unsigned u = 10;
int s = -1;
if (s < u) // s 被转为 unsigned, 变成很大的数
 std::cout << "true\n"; // 不会执行!
```

```
// 正确写法:
if (s < static_cast<int>(u)) // 显式转换
// 或避免混用
```

```
// ===== 错误19: 隐式转换丢失精度 =====
// 错误代码:
double d = 3.14;
int x = d; // 警告: 精度丢失

// 正确写法:
int x = static_cast<int>(d); // 显式转换, 意图明确
```

```
// ===== 错误20: C 风格转换 =====
// 错误代码:
int *p = (int *)malloc(10); // C 风格转换太粗暴
Derived *d = (Derived *)base; // 不安全

// 正确写法:
int *p = static_cast<int *>(malloc(10));
Derived *d = dynamic_cast<Derived *>(base); // 运行时常检查
```

23. 重要表格汇总
23.1 内置类型速查

| 类型                 | 大小 (通常) | 范围                                    | 格式符 scanf / printf |
|--------------------|---------|---------------------------------------|--------------------|
| short              | 2 bytes | -32,768 ~ 32,767                      | %hd                |
| unsigned short     | 2 bytes | 0 ~ 65,535                            | %hu                |
| int                | 4 bytes | -2 <sup>31</sup> ~ 2 <sup>31</sup> -1 | %d / %u            |
| unsigned int       | 4 bytes | 0 ~ 2 <sup>32</sup> -1                | %u                 |
| long long          | 8 bytes | -2 <sup>63</sup> ~ 2 <sup>63</sup> -1 | %lld               |
| unsigned long long | 8 bytes | 0 ~ 2 <sup>64</sup> -1                | %llu               |
| float              | 4 bytes | ±3.4e38                               | %f                 |
| double             | 8 bytes | ±1.8e308                              | %lf / %f           |
| char               | 1 byte  | -128 ~ 127 或 0 ~ 255                  | %c                 |
| bool               | 1 byte  | true/false                            | %d (C)             |

### 23.2 运算符优先级（从高到低）

| 优先级 | 运算符                             | 说明      |
|-----|---------------------------------|---------|
| 1   | ::                              | 作用域解析   |
| 2   | ++a --a type() type{} a() a[] . | 后缀/成员访问 |
| 3   | ++a --a +a -a ! ~ * & sizeof    | 前缀/一元   |
| 4   | . * ->                          | 成员指针    |
| 5   | a*b a/b a%b                     | 乘除模     |
| 6   | a+b a-b                         | 加减      |
| 7   | << >>                           | 位移      |
| 8   | < <= > >=                       | 比较      |
| 9   | == !=                           | 相等      |
| 10  | &                               | 按位与     |
| 11  | ~                               | 按位异或    |
| 12  |                                 | 按位或     |
| 13  | &&                              | 逻辑与     |
| 14  |                                 | 逻辑或     |
| 15  | a?:b:c                          | 三元条件    |
| 16  | = += 等                          | 赋值      |
| 17  | throw                           | 异常      |
| 18  | :                               | 造号      |

### 23.3 C vs C++ 对比

| 特性       | C              | C++               |
|----------|----------------|-------------------|
| 函数重载     | 不支持            | 支持                |
| 默认参数     | 不支持            | 支持                |
| 引用       | 不支持            | 支持                |
| 类和对象     | 不支持（只有 struct） | 完整支持              |
| 函数/运算符重载 | 不支持            | 支持                |
| 模板       | 不支持            | 支持                |
| 匿名空间     | 不支持            | 支持                |
| 异常处理     | 有限             | 完整支持              |
| bool 类型  | C99 起          | 原生支持              |
| 内存管理     | malloc/free    | new/delete + 智能指针 |
| 输入输出     | printf/scanf   | iostream          |
| 字符串      | char*          | std::string       |
| 泛型编程     | void*          | 模板                |
| 标准库      |                | STL（丰富）           |

### 23.4 STL 容器复杂度对比

| 容器                | 访问       | 插入（中间）   | 删除       | 查找       |
|-------------------|----------|----------|----------|----------|
| vector            | O(1)     | O(1)     | O(n)     | O(n)     |
| deque             | O(1)     | O(1)     | O(n)     | O(n)     |
| list              | O(n)     | O(1)     | O(1)     | O(n)     |
| forward_list      | O(1)     | O(1)     | O(1)     | O(n)     |
| set/map           | O(log n) | O(log n) | O(log n) | O(log n) |
| unordered_set/map | O(1)     | O(1)     | O(1)     | O(1)     |

### 23.5 拷贝控制成员默认行为

| 函数           | 默认行为      | 何时需要自定义      |
|--------------|-----------|--------------|
| 默认构造函数       | 成员逐个默认初始化 | 需要特定默认状态     |
| 析构函数         | 成员逐个销毁    | 需要释放资源       |
| 拷贝构造函数       | 成员逐个拷贝    | 需要深拷贝        |
| 拷贝赋值         | 成员逐个赋值    | 需要深拷贝/自我赋值安全 |
| 移动构造 (C++11) | 成员逐个移动    | 需要高效转移资源     |
| 移动赋值 (C++11) | 成员逐个移动    | 需要高效转移资源     |

### 23.6 指针、引用、值传递对比

| 特性          | 传值          | 传指针          | 传引用          | 传 const 引用         |
|-------------|-------------|--------------|--------------|--------------------|
| 语法          | void f(T x) | void f(T *x) | void f(T &x) | void f(const T &x) |
| 能否修改原值      | 否           | 是（通过解引用）     | 是            | 否                  |
| 能否传 nullptr | N/A         | 可以           | 不可以          | 不可以                |
| 拷贝开销        | 有（拷贝整个对象）   | 无（只拷贝指针）     | 无            | 无                  |
| 空值处理        | 对象总是有效      | 需要检查         | 总是有效         | 总是有效               |
| 适用场景        | 小对象         | C 风格/可选参数    | C++ 修改参数     | C++ 只读对象           |

### 24. 考试/作业常用代码模板

#### 24.1 快速输入输出模板

```
#include <bits/stdc++.h>
using namespace std;

int main() {
 ios::sync_with_stdio(false);
 cin.tie(nullptr);

 int n;
 cin >> n;
 // ...

 return 0;
}
```

#### 24.2 读取到 EOF 模板

```
// 方式1: while 循环
int x;
while (cin >> x) {
 // 处理 x
}

// 方式2: for 循环
for (int x; cin >> x;) {
 // 处理 x
}
```

#### 24.3 排序与去重模板

```
vector<int> v(n);
// ... 填充数据 ...

sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
// 现在 v 是有序且无重复元素的
```

#### 24.4 二分查找模板

```
// lower_bound: 第一个 >= target 的位置
auto it = lower_bound(v.begin(), v.end(), target);

// upper_bound: 第一个 > target 的位置
auto it = upper_bound(v.begin(), v.end(), target);

// 判断元素是否存在
bool exists = binary_search(v.begin(), v.end(), target);

// 获取元素个数（在已排序且有重复的数组中）
int count = upper_bound(v.begin(), v.end(), target) - lower_bound(v.begin(), v.end(), target);
```

#### 24.5 大数取模模板

```
const int MOD = 1e9 + 7;

long long mod_pow(long long base, long long exp) {
 long long result = 1;
 base %= MOD;
 while (exp > 0) {
 if (exp & 1) result = result * base % MOD;
 base = base * base % MOD;
 exp >>= 1;
 }
 return result;
}
```

#### 24.6 并查集模板

```
class UnionFind {
public:
 vector<int> parent, rank;

 UnionFind(int n) : parent(n), rank(n, 0) {
 iota(parent.begin(), parent.end(), 0);
 }

 int find(int x) {
 return parent[x] == x ? x : parent[x] = find(parent[x]);
 }
}
```

```
bool unite(int x, int y) {
 x = find(x); y = find(y);
 if (x == y) return false;
 if (rank[x] < rank[y]) swap(x, y);
 parent[y] = x;
 if (rank[x] == rank[y]) rank[x]++;
 return true;
}

bool same(int x, int y) { return find(x) == find(y); }
```

### 24.7 完整类定义模板（Rule of Five）

```
#include <algorithm>
#include <cstring>
#include <utility>

class MyClass {
private:
 // 数据成员
 std::size_t size_;
 int *data_;

public:
 // 构造函数
 explicit MyClass(std::size_t n) : size_(n), data_(new int[n]{}) {}

 // 默认构造函数
 MyClass() : MyClass(0) {}

 // 拷贝构造函数
 MyClass(const MyClass &other) : size_(other.size_), data_(new int[other.size_] {
 std::copy_n(other.data_, size_, data_);
 }) {}

 // 移动构造函数
 MyClass(MyClass &other) noexcept : size_(std::exchange(other.size_, 0)), data_(std::exchange(other.data_, nullptr)) {}

 // 拷贝/移动赋值运算符（copy-and-swap）
 MyClass &operator=(MyClass other) noexcept {
 swap(other);
 return *this;
 }

 // 析构函数
 ~MyClass() { delete[] data_; }

 // swap
 void swap(MyClass &other) noexcept {
 using std::swap;
 swap(size_, other.size_);
 swap(data_, other.data_);
 }

 // 元素访问
 int &operator[](std::size_t i) { return data_[i]; }
 const int &operator[](std::size_t i) const { return data_[i]; }

 // 常量
 std::size_t size() const noexcept { return size_; }
 bool empty() const noexcept { return size_ == 0; }
};

void swap(MyClass &a, MyClass &b) noexcept { a.swap(b); }
```

### 24.8 带比较器的排序模板

```
// 结构体排序
struct Student {
 string name;
 int score;
};

// 按分数降序，分数相同按名字升序
sort(students.begin(), students.end(), [](const Student &a, const Student &b) {
 if (a.score != b.score) return a.score > b.score;
 return a.name < b.name;
});
```

### 24.9 map 计数模板

```
// 统计频率
map<string, int> freq;
for (const auto &word : words) {
 freq[word]++;
}

// 找最大值
string max_word;
int max_count = 0;
for (const auto &[word, count] : freq) {
 if (count > max_count) {
 max_count = count;
 max_word = word;
 }
}
```

### 24.10 文件读写模板

```
#include <fstream>
#include <sstream>

// 读文件
ifstream infile("input.txt");
if (!infile) {
 cerr << "Cannot open input.txt\n";
 return 1;
}
string line;
while (getline(infile, line)) {
 istringstream iss(line);
 int a, b;
 iss >> a >> b;
 // 处理
}
```

```
// 写文件
ofstream outfile("output.txt");
outfile << "Result: " << result << "\n";
// 文件在 ofstream 析构时自动关闭
```

### 附录

### A. 头文件速查

| 头文件                | 内容                                  |
|--------------------|-------------------------------------|
| <iostream>         | std::cin, std::cout, std::cerr      |
| <iomanip>          | 格式化输出 (setw, setprecision)          |
| <fstream>          | 文件流 (ifstream, ofstream)            |
| <sstream>          | 字符串流 (istringstream, ostringstream) |
| <string>           | std::string                         |
| <vector>           | std::vector                         |
| <list>             | std::list                           |
| <forward_list>     | std::forward_list                   |
| <deque>            | std::deque                          |
| <map>              | std::map, std::multimap             |
| <set>              | std::set, std::multiset             |
| <unordered_map>    | std::unordered_map                  |
| <unordered_set>    | std::unordered_set                  |
| <algorithm>        | STL 算法 (sort, find, etc.)           |
| <numeric>          | 数值算法 (accumulate, etc.)             |
| <iterator>         | 迭代器辅助 (back_inserter, etc.)         |
| <memory>           | 智能指针 (unique_ptr, shared_ptr)       |
| <utility>          | pair, make_pair, move, swap         |
| <tuple>            | tuple, make_tuple                   |
| <functional>       | 函数对象, greater, less                 |
| <cmath>            | 数学函数 (sqrt, pow, etc.)              |
| <cctype>           | 字符处理 (isdigit, isalpha, etc.)       |
| <cstring>          | C 字符串函数 (strlen, strcpy)            |
| <cstdlib>          | malloc, free, atoi, exit            |
| <ctime>            | 时间函数                                |
| <cassert>          | assert 宏                            |
| <type_traits>      | 类型特征 (C++11)                        |
| <initializer_list> | 初始化列表 (C++11)                       |

### B. C++ 标准版本特性速查

| 版本    | 年份   | 主要特性                                                 |
|-------|------|------------------------------------------------------|
| C++98 | 1998 | 首个标准化版本                                              |
| C++03 | 2003 | 小修訂                                                  |
| C++11 | 2011 | auto, nullptr, lambda, 智能指针, 范围 for, 移动语义, constexpr |
| C++14 | 2014 | 泛型 lambda, auto 返回值, make_unique                     |
| C++17 | 2017 | if constexpr, 结构化绑定, optional, string_view, CTAD     |
| C++20 | 2020 | concept, ranges, coroutine, module, ...              |
| C++23 | 2023 | deducing this, std::print, 更多 constexpr              |

CS100 课程基于 C++17 标准。

### C. 推荐的编译选项

• 推荐（用于作业和考试）  
g++ -std=c++17 -Wall -Wextra -O2 program.cpp -o program

• 调试模式  
g++ -std=c++17 -g -O0 program.cpp -o program

• 严格模式（捕获更多错误）  
g++ -std=c++17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion program.cpp -o program

地址和解引用  
核心概念取地址：拿到变量的牌门号 • 解引用\*：通过牌门号打开房间

| 操作 | 输入 → 输出      | 本质   |
|----|--------------|------|
| &x | 变量 T → 指针 T* | 取门牌号 |
| *p | 指针 T* → 变量 T | 开门取值 |

声明 vs 表达式  
\* 和 & 身兼数职：声明中是形容词，表达式中是动词。

| 上下文  | * 含义       | & 含义     |
|------|------------|----------|
| 声明中  | “指向...的指针” | “...的引用” |
| 表达式中 | 解引用，取出值    | 取地址，拿到地址 |

### 组合运算速查

| 表达式  | 类型推导            | 结果       | 合法 |
|------|-----------------|----------|----|
| &x   | T → T*          | x 的地址    | ✓  |
| *p   | T* → T          | p 指向的对象  | ✓  |
| *&x  | & → T* → T*     | x 本身（抵消） | ✓  |
| *p   | ** → T → & → T* | 等价于 p    | ✓  |
| &&x  | --              | 右值无法取址   | ✗  |
| **pp | ** → T* → T     | 二级解引用    | ✓  |

### 参数匹配表

| #  | 函数声明                  | 调用代码               | 结果 | 关键解析                                 |
|----|-----------------------|--------------------|----|--------------------------------------|
| 1  | void f(int a[100])    | int a[20]; f(a);   | OK | 形参退化为 int*, 长度不检查                    |
| 2  | void f(int (&a)[100]) | int a[100]; f(&a); | CE | &a 是 int(*)[100], 但形参要引用 int(&)[100] |
| 3  | void f(int (&a)[100]) | int a[100]; f(*a); | CE | *a 得 int, 形参要 int(*)[100]            |
| 4  | void f(int (&a)[100]) | int a[100]; f(*a); | OK | &a 类型正是 int(&)[100]                  |
| 5  | void f(int (&a)[100]) | int a[100]; f(a);  | OK | a 左值绑定到 int(&)[100]                  |
| 6  | int *p = &a[0];       | —                  | OK | &a[0] 等价 a, 类型 int*                  |
| 7  | int (*p)[100] = &a;   | —                  | OK | &a 是数组指针, 非 int*                     |
| 8  | int * &rp = p;        | —                  | OK | rp 是指针的引用                            |
| 9  | int &*pr;             | —                  | CE | C++ 禁止指向引用的指针                        |
| 10 | int (&rp)[100]        | —                  | CE | a 退化为 int*, 不匹配                      |
| 11 | int *p; *p = 10;      | —                  | UB | p 未初始化, 解引用是未定义行为                    |
| 12 | int &r = *p;          | —                  | OK | *p 是左值, 可绑定引用 (需 p 有效)               |
| 13 | int *p = &r;          | —                  | OK | &r 得到 r 所绑对象的地址                      |

### 判断口诀

- 先看上下文：声明里还是表达式里？
- 声明中：从变量名往左读：\* = 指针，& = 引用
- 表达式中：& = 取地址，\* = 解引用
- 优先级：\* 和 & 优先级相同，右结合：声明用括号改顺序

### 类型对照速记

| 写法                  | 实际类型             | 调用时传入        |
|---------------------|------------------|--------------|
| void f(int a[N])    | int*（长度丢失）       | 任意 int* 或数组名 |
| void f(int (&a)[N]) | int (&)[N]（保留长度） | 数组名本身 f(a)   |
| void f(int (&a)[N]) | int (*)[N]（数组指针） | 数组地址 f(&a)   |

### 常见错误模式

| 错误写法          | 错误原因       | 正确写法             |
|---------------|------------|------------------|
| int &*p;      | 不能指向引用     | int *p = ...;    |
| int *p; &*p;  | *p 是左值但无意义 | p = &x;          |
| f(&a) 给引用参数   | 引用要对象，不要地址 | f(a)             |
| f(a) 给数组指针参数  | 数组名退化为元素指针 | f(&a)            |
| f(*a) 给数组指针参数 | *a 是首元素    | f(&a) 或 f(a) 给引用 |

### 操作符值类别速查

| 表达式                  | 值类别    | 原因                     |
|----------------------|--------|------------------------|
| int a; a             | 左值     | 变量有名字、有地址              |
| 42, 'c', 3.14        | 纯右值    | 字面常量，无地址               |
| a + b                | 纯右值    | 任意 int* 或数组名           |
| std::string("hello") | 左值     | 临时变量                   |
| ++i (前置)             | 左值     | 返回自身引用                 |
| i++ (后置)             | 纯右值    | 返回旧值的临时副本              |
| --i                  | 左值     | 同 ++i                  |
| i--                  | 纯右值    | 同 i++                  |
| *ptr                 | 左值     | 解引用 == 所指对象本身          |
| &a                   | 纯右值    | 取地址值是临时值               |
| &*ptr                | 纯右值    | *ptr 左值，但 & 产生地址右值     |
| a[i]                 | 左值     | 等价 *(a+i)，返回元素本身       |
| (a, b)               | 看 b    | 取决于 b 的值类别             |
| a ? b : c            | 看 b, c | 同类型且都左值 → 左值；否则右值      |
| static_cast<T&>(a)   | 左值     | 显式左值引用                 |
| static_cast<T&&>(a)  | 将亡值    | 强制转右值引用                |
| std::move(a)         | 将亡值    | 等价 static_cast<T&&>(a) |
| T&& 形参在函数体内          | 左值     | “临时”右值的右值引用是左值！        |
| 函数返回 T（非引用）          | 纯右值    | 返回副本/临时对象              |
| 函数返回 T&              | 左值     | 返回引用                   |
| 函数返回 T&&             | 将亡值    | 返回右值引用                 |

| 表达式                        | 值类别 | 详细解释                       |
|----------------------------|-----|----------------------------|
| p                          | 左值  | 指针变量本身                     |
| *p                         | 左值  | 解引用 == p 指向的那个对象（即 x 本身）   |
| &x                         | 纯右值 | 取地址运算结果 == 地址数值，临时值        |
| *p                         | 纯右值 | *p 是左值，& 取其地址 → 地址值是临时值    |
| *&x                        | 左值  | &x 是右值（地址），* 解引用 → 回到 x 本身 |
| p + 1                      | 纯右值 | 指针算术产生新地址值                 |
| *(p + 1)                   | 左值  | 解引用新地址 → 该地址上的对象           |
| 表达式                        | 值类别 | 详细解释                       |
| for (auto var : vs)        | 左值  | 每次循环拷贝一个 std::string       |
| for (auto &var : vs)       | 左值  | 引用原对象，可修改                  |
| for (const auto &var : vs) | 左值  | 引用原对象，只读                   |

| 写法                         | 实际行为                 | 代价            |
|----------------------------|----------------------|---------------|
| for (auto var : vs)        | 每次循环拷贝一个 std::string | O(n) 深拷贝，内存分配 |
| for (auto &var : vs)       | 引用原对象，可修改            | 零拷贝，O(1)      |
| for (const auto &var : vs) | 引用原对象，只读             | 零拷贝，O(1)，安全   |

